“MEMORY CARD”

PROYECTO DE DESARROLLO SOFTWARE

CICLO FORMATIVO DE GRADO SUPERIOR DE DESARROLLO DE APLICACIONES MULTIPLATAFORMA

ADRIÁN CID, ADRIÁN JUBERA Y RODRIGO SOSA
JAVIER GONZÁLEZ SANTURDE

Curso 2025-2026

ÍNDICE

1. Resumen. 4
2. Abstract. 5
3. Introducción. 6
3.1. Análisis Cuantitativo de la Producción de Videojuegos 6
3.2. El Desafío de la Visibilidad del Inventario 6
3.3. Justificación de la Experiencia de Usuario (UX) en Entornos Saturados 7
3.4. Relevancia Social y Personal del Proyecto 7
4. Justificación y Objetivos. 8
4.1. Justificación y motivos 8
4.2. Objetivo general del proyecto 8
4.3. Objetivos específicos en el proyecto 8
5. Material y Método. 11
5.1. Materiales: Recursos Técnicos 11
5.2. Metodología del Proyecto 12
6. Resultados/Contenido. 14
6.1. Fase de Análisis y Diseño del Sistema 14
6.2. Desarrollo del Core: Backend y Base de Datos 15
6.3. Desarrollo de Clientes: Web y Móvil 16
6.3.1. Cliente Web Front-end: React 17
6.3.2. Aplicación Móvil Nativa: Android 18
6.4. Integración, Pruebas y Sincronización 19
7. Conclusiones y discusión. 22
7.1. Aportación del proyecto 22
7.2. Reflexión técnica: Discusión 22
7.3. Relación con lo aprendido en el Grado 23
7.4. Trabajo futuro y líneas de mejora 24
8. Bibliografía. 26
9. Anexos 28
Anexo 1. Arquitectura de Datos y Funcional 28
Anexo 2. Especificación Técnica de Funcionalidades 30
Bloque A: Gestión de Accesos 30
Bloque B: Gestión de la Colección 43
Bloque C: Interacción Social 55
Anexo 3. Diseño Visual y Experiencia de Usuario (UX/UI) 65
Anexo 4. Implementación y Sincronización 67
- Figura 1. Interfaz del Cliente Web (React) - Perfil. 67
- Figura 2. Interfaz del Cliente Web (React) - Inicio. 67
- Figura 3. Interfaz del Cliente Web (React) - Juego. 68

- Figura 4. Interfaz del Cliente Web (React) - Amigos..... 68
- Figura 5. Interfaz del Cliente Web (React) – Inicio de Sesión..... 69
- Figura 6. Interfaz del Cliente Web (React) - Registro. 69
- Figura 7. Inicio de sesión en la Aplicación Móvil Nativa (Android). 70
- Figura 8. Página principal en la Aplicación Móvil Nativa (Android). 70
- Figura 9. Menú desplegable lateral en la Aplicación Móvil Nativa (Android). 71
- Figura 10. Sección principal del perfil en la Aplicación Móvil Nativa (Android). 71
- Figura 11. Sección de Biblioteca del perfil en la Aplicación Móvil Nativa (Android). 72
- Figura 12. Sección de reviews del perfil en la Aplicación Móvil Nativa (Android). 72
- Figura 13. Sección de Lista de deseos del perfil en la Aplicación Móvil Nativa (Android). 73
- Figura 14. Sección de listas personalizadas del perfil en la Aplicación Móvil Nativa (Android). 73

1. Resumen.

Este Trabajo de Fin de Grado se detalla el proceso de desarrollo de la aplicación "Memory Card", un sistema de gestión, organización y red social relacionada con el mundo de los videojuegos.

El objetivo principal al que quiere aspirar este proyecto es ofrecer a los usuarios un espacio unificado e intuitivo donde puedan registrar, agrupar, valorar y reseñar los diferentes productos, dando solución al problema planteado de la excesiva cantidad de juegos repartidos entre las múltiples plataformas digitales y físicas, ya que esta situación conlleva un descontrol en el seguimiento de los títulos que los usuarios ya poseen.

Para darle solución a esta controversia, se ha planteado y ejecutado una arquitectura robusta, con un diseño amigable para que cualquier usuario pueda seguir el flujo de la aplicación y una seguridad excepcional residida en una base de datos relacional.

Utilizando la misma base de datos, se han construido desde cero dos entornos independientes: por un lado, una página web moderna, utilizando la librería React.js combinada con Vite; y, por otro lado, una aplicación móvil nativa para dispositivos Android programada en Java con un maquetado en XML. Estos dos productos comparten el mismo tipo de interfaz, pero con los cambios necesarios para los distintos dispositivos.

En cuanto al método de trabajo, este proyecto se ha desarrollado bajo una metodología de trabajo ágil basada en la asignación de Sprints semanales, garantizando un flujo de trabajo colaborativo mediante el uso de GitHub para el control de versiones, Figma para el prototipado de interfaces y Notion para la coordinación y asignación de tareas del equipo.

PALABRAS CLAVE: React, Android, XML, Sprints, GitHub, Figma, Notion.

2. Abstract.

This Final Degree Project details the development process of the "Memory Card" application, a management, organization, and social networking system related to the world of video games.

The main objective of this project is to offer users a unified and intuitive space where they can register, group, rate, and review different products, addressing the problem of the excessive number of games distributed across multiple digital and physical platforms. This situation leads to a lack of control in tracking the titles that users already own.

To resolve this issue, a robust architecture has been designed and implemented, with a user-friendly design so that any user can follow the application's flow, and exceptional security based on a relational database.

Using the same database, two independent environments have been built from scratch: a modern website using the React.js library combined with Vite; and a native mobile application for Android devices programmed in Java with an XML layout. These two products share the same type of interface, but with the necessary modifications for different devices.

Regarding the workflow, this project was developed using an agile methodology based on weekly sprints, ensuring a collaborative workflow through the use of GitHub for version control, Figma for interface prototyping, and Notion for team coordination and task assignment.

KEY WORDS: React, Android, XML, Sprints, GitHub, Figma, Notion.

3. Introducción.

3.1. Análisis Cuantitativo de la Producción de Videojuegos

La premisa de que el videojuego es el hobby principal de una gran parte de la población actual no es solo una percepción subjetiva. Para comprender la necesidad de una herramienta como Memory Card, es imperativo analizar la magnitud del mercado actual. Según los registros históricos de **SteamDB (2026)**, solo la plataforma líder de distribución en PC ha experimentado una aceleración sin precedentes en el número de lanzamientos anuales. Mientras que en la década pasada los lanzamientos se contaban por cientos, en el periodo comprendido entre 2022 y 2025 se ha consolidado una tendencia de crecimiento que supera los 14,000 títulos nuevos por año, alcanzando picos de casi 20,000 lanzamientos anuales al cierre del último ejercicio y en curso del año actual la cifra ya gira en torno a los casi 7,000 títulos.

Esta explosión cuantitativa ha llevado al catálogo acumulado de Steam a superar la barrera de los 125,000 productos activos. Este escenario genera un entorno de saturación donde el usuario medio se ve incapaz de monitorizar, organizar o recordar la totalidad de las experiencias a las que tiene acceso, justificando el desarrollo de sistemas de gestión de bibliotecas personales que centralicen esta información.

3.2. El Desafío de la Visibilidad del Inventario

El problema de la gran cantidad de videojuegos existentes no radica únicamente en la producción, sino en la pérdida de visibilidad de los títulos dentro de las bibliotecas de los propios usuarios. Los datos de **SteamDB (2026)** reflejan que una fracción significativa del catálogo disponible en las cuentas de los jugadores permanece en un estado de inactividad técnica. Este fenómeno es una consecuencia directa de las rebajas estacionales y la adquisición de lotes de software masivos (como los que ofrece Humble Bundle).

Memory Card surge como una respuesta técnica a este desorden digital. La aplicación propone un espacio de "curación personal", donde el usuario no es un mero receptor pasivo de software, sino un gestor activo de su colección. Al permitir la valoración, el agrupamiento y la reseña, la aplicación mitiga el efecto de la "biblioteca olvidada", asegurando que el flujo de juegos pendientes (backlog) sea manejable y visualmente intuitivo. La arquitectura de la aplicación, enfocada en un

diseño limpio, busca precisamente contrarrestar el ruido visual que impera en las tiendas digitales actuales.

3.3. Justificación de la Experiencia de Usuario (UX) en Entornos Saturados

En un mercado donde se publican decenas de juegos al día, la simplicidad se convierte en una ventaja competitiva. La literatura sobre usabilidad en aplicaciones móviles de gestión de datos (**Yablonski, s.f.**) subraya que, ante bases de datos masivas, la interfaz debe priorizar la reducción de la carga cognitiva. La propuesta de **Memory Card** se alinea con estas necesidades mediante herramientas de filtrado avanzado.

No se trata solo de buscar un juego por su título, sino de permitir que el usuario redescubra su propia colección basándose en sus gustos y estados de finalización. La integración de un flujo de usuario sencillo permite que el registro de una nueva experiencia (o el rescate de una antigua de plataformas como Humble Bundle o GOG) sea un proceso rápido, evitando que la propia herramienta de organización se convierta en una carga adicional para el jugador.

3.4. Relevancia Social y Personal del Proyecto

Finalmente, el desarrollo de esta aplicación responde a una motivación antropológica: la identidad del jugador moderno. Como indican **Fernández Ruiz et al. (2022)**, el videojuego ha pasado de ser un producto de consumo a un pilar de la personalidad de las nuevas generaciones. Crecer jugando implica acumular una biografía lúdica que merece ser preservada y organizada. Memory Card no es solo una base de datos; es un diario de experiencias que reconoce que el videojuego es, para muchos, un elemento necesario en su desarrollo personal y social.

4. Justificación y Objetivos.

4.1. Justificación y motivos

Vemos que en la industria del videojuego no existe una aplicación ya consolidada donde usuarios puedan compartir de forma sencilla un seguimiento de lo que juegan y mucho menos, compartirlo con otros usuarios, a diferencia del cine, con la ya conocida y consolidada “**Letterboxd**”. Esto nos lleva a que personalmente, nuestros grupos de amigos tengan cada uno una aplicación o método diferente para llevar a cabo esa tarea, ya que ninguna de las existentes se use tanto en móvil como en web o sea lo suficientemente conocida por todos estos.

A nivel profesional, este Trabajo de Fin de Grado (TFG), es una gran oportunidad para simular un entorno de desarrollo de software real, que nos permita hacer frente a dos campos diferentes pero entrelazados que son el desarrollo web y el desarrollo de aplicaciones móviles. Al ser un grupo conformado por tres integrantes, nuestra mayor motivación es el reto de una integración de sistemas heterogénea; ya que no se trata de programar en tres campos diferentes (Java para móvil, React para Web y PHP para el backend), sino hacer que los tres estén coordinados para que la misma información se pueda compartir entre diferentes sistemas y sigan funcionando y coordinándose conjuntamente una vez desplegada la plataforma en la red con los servidores que hacen funcionar el backend en la nube.

4.2. Objetivo general del proyecto

Diseñar, desarrollar y desplegar un sistema de información multiplataforma y distribuido para la gestión social de videojuegos, que permita a los usuarios llevar un registro personal, compartir información con otros usuarios y consultar información en tiempo real, manteniendo la consistencia de los datos entre una web y una aplicación móvil mediante un backend distribuido en la web.

4.3. Objetivos específicos en el proyecto

Los objetivos específicos de este TFG están organizados en diferentes niveles:

A nivel organizativo:

- **Trabajar** con un flujo colaborativo basado en el control de versiones (GIT), usando metodología SCRUM en la asignación de tareas, para su fácil revisión e implementación.
- **Documentar** el desarrollo del software, incluyendo especificaciones para un mantenimiento y trabajo futuro, además de diagramas de flujo, casos de uso y entidad-relación.
- **Garantizar** un flujo de desarrollo paralelo y organizado mediante el uso de ramas (*branches*) en GitHub, permitiendo la implementación de nuevas funcionalidades de forma aislada y segura sin comprometer la estabilidad del código principal.

A nivel técnico:

- **Modelar** un esquema de datos relacional robusta mediante el sistema de gestión **MySQL**, garantizando una arquitectura escalable que soporte de manera eficiente el flujo de datos y asegure la integridad de la información del proyecto.
- **Implementar** una **API RESTful** a través de PHP, definiendo *endpoints* claros y seguros para permitir la comunicación bidireccional (lectura y escritura) desde los clientes web y móvil.
- **Asegurar** la integridad y persistencia de la información, garantizando que las modificaciones realizadas en cualquiera de los clientes se reflejen inmediatamente en el otro.
- **Desarrollar** una aplicación nativa para dispositivos Android escrita en Java que aproveche los recursos del sistema, ofrezca una navegación fluida y se adapte a los elementos táctiles de los dispositivos móviles.
- **Articular** una interfaz web responsive mediante React que priorice la experiencia del usuario (UX), facilitando así la navegación y gestión de grandes índices de información.
- **Desplegar** la base de datos en un entorno remoto para garantizar la integridad y persistencia de la información, evitando las limitaciones de sincronización de los soportes físicos.
- **Alojar** el backend en un servidor en la nube para asegurar la disponibilidad continua del servicio y eliminar la dependencia de una infraestructura local.

A nivel de diseño:

- **Estructurar** una interfaz de usuario minimalista y diáfana que actúe como guía visual para el usuario, optimizar los flujos de navegación para asegurar que la interacción con la plataforma sea fluida, coherente y carente de ambigüedades y utilizar simbología universal para la correcta distinción de las distintas pantallas de la aplicación.
- **Componer** la estética de la aplicación mediante una paleta de colores sobria y una tipografía acorde a la identidad del proyecto, priorizando el contraste necesario para resaltar los elementos clave, como las caratulas y los componentes interactivos.
- **Diseñar** una arquitectura visual basada en componentes homólogos que, pese a las diferencias tecnológicas entre el marcado XML de Android y el JSX de React, mantengan una estética unificada y una experiencia de usuario (UX) consistente en ambas plataformas.

5. Material y Método.

En este apartado detallamos tanto las herramientas técnicas que estamos usando como la forma en la que nos hemos organizado para trabajar en equipo.

5.1. Materiales: Recursos Técnicos

Para montar todo el sistema de Memory Card, hemos seleccionado las siguientes tecnologías:

- **Para el Backend:** La base de datos corre sobre MySQL, que es donde guardamos toda la información de la plataforma. Estamos trabajando con PHP 8.x y un servidor Apache para el desarrollo y las pruebas en local. Para su despliegue hemos optado por usar el PaaS Railway para alojar la base de datos, que es compatible con la semántica de MySQL y para toda la lógica de las llamadas API se está usando Render.
- **Para la App Móvil:** Usamos Android Studio y programamos en Java, aprovechando las herramientas del SDK de Android para que la aplicación sea nativa.
- **Para la Web:** Como entorno de desarrollo hemos elegido Visual Studio Code. Vamos a usar React.js junto con Vite. Básicamente, hemos elegido React porque nos facilita mucho organizar toda la interfaz por piezas y que la web se sienta "viva" cuando el usuario interactúa con ella. Usamos Vite porque, a la hora de programar, es mucho más rápido y ligero que lo que se usaba antes, así que nos ahorra tiempo y líos al montar el proyecto. Queremos que la experiencia de navegar por la colección de juegos sea más agradable.

Gestión y Diseño:

- Git y GitHub: Fundamental para llevar un control de los cambios y no pisarnos el código con la separcion en ramas del proyecto, además de la función de GitHub Pages para el despliegue de la interfaz web.
- Notion: Lo usamos como centro de organización en el que estarán definidas las tareas de cada uno, definidas en SPRINTS con la duración de ambos, tenemos apartados de brainstorming, investigación y posibles vías de desarrollo altermativas.

- Figma: Aquí es donde hemos diseñado cómo se va a ver la aplicación y la web antes de ponernos a programar.
- Discord: Es la aplicación que hemos elegido para llevar a cabo nuestras propias reuniones de seguimiento.
- Draw.io: Es la web y extensión de Visual Studio Code que hemos usado para dibujar los diagramas que nos han llevado a una comprensión clara y directa del funcionamiento de la aplicación.

5.2. Metodología del Proyecto

Nos hemos organizado con una metodología Ágil basada en SCRUM, dividida por Sprints. Al ser tres personas, esto nos permite dividir el trabajo en bloques, tener cada uno las tareas definidas y ver avances cada poco tiempo. El plan de trabajo sigue estos pasos:

1. **Análisis y Diseño:** Empezamos aclarando qué funciones queríamos y cómo deberían funcionar (requisitos), montando el diagrama de la base de datos y haciendo los diseños en Figma.
2. **Desarrollo del Core (Backend):** Es el primer paso porque sin la base de datos y la API funcionando, ni la web ni la aplicación tendrían de dónde sacar la información.
3. **Desarrollo en Paralelo de los clientes:** Una vez que los *endpoints* (las rutas de la API) están listos, los encargados de **Android** y **React** pueden avanzar a la vez conectando las interfaces con los datos reales que vienen del backend.
4. **Integración y Pruebas:** Aquí es donde juntamos todo. Probamos que, si alguien añade un juego desde el móvil, aparezca al segundo en la web para asegurar que la información no se pierda.
5. **Despliegue:** Una vez todo funciona correctamente en local, se aloja la base de datos en Render, la lógica del backend en Railway, estableciendo una conexión entre ambos. Posteriormente se aloja la interfaz web en la sección de GitHub Pages asociada al repositorio y se hace una build apk de la aplicación Android para usar en dispositivos móviles.
6. **Documentación:** Realmente este paso se hace a la vez que cada uno de los anteriores.

- **Entorno de Ejecución:** Dejamos apuntado qué versiones de PHP y MySQL estamos usando para que, el día de la presentación, sepamos exactamente qué configuración necesita el servidor para que todo arranque sin fallos.
- **Protocolo de Pruebas:** No nos limitamos a programar; también hacemos un listado de pruebas (como crear usuarios de prueba o escribir reseñas) para confirmar que la comunicación entre el backend y los clientes es estable y no da errores.

6. Resultados/Contenido.

Tras completar el ciclo de desarrollo bajo el marco de trabajo **Scrum**, se ha consolidado un sistema de gestión de información distribuido y multiplataforma denominado **"Memory Card"**. La ejecución del proyecto se ha articulado mediante una gestión ágil en la plataforma **Notion**, la cual permitió centralizar el flujo de trabajo a través de la planificación de *sprints* periódicos y la asignación detallada de tareas, mientras que para los aspectos de la programación se ha empleado **GitHub(s.f.)**.

Siguiendo las directrices de **Schwaber y Sutherland (2020)** en *La Guía de Scrum*, el proceso se basó en la inspección y adaptación continua, validando al cierre de cada iteración si los objetivos establecidos habían sido alcanzados. Estas sesiones de revisión no solo permitieron auditar el progreso técnico, sino que fueron fundamentales para definir estratégicamente las prioridades del siguiente *sprint*. A continuación, se detallan los resultados y conclusiones derivados de cada una de las fases cronológicas que dieron forma al proyecto:

6.1. Fase de Análisis y Diseño del Sistema

En esta primera fase, se definió tanto la estructura lógica como la identidad visual que sostiene el sistema de gestión. Los resultados se dividen en dos pilares fundamentales:

Arquitectura Funcional y de Datos

Se modeló una arquitectura de datos relacional mediante el sistema de gestión **MySQL (2023)**, seleccionada específicamente por su robustez y compatibilidad con el esquema diseñado para la gestión de usuarios y el catálogo global de videojuegos **(ver Figura 1 en Anexo 1)**. Esta elección se fundamentó en la necesidad de garantizar la integridad referencial de la información, optando por este sistema tras descartar modelos no relacionales debido a la naturaleza estructurada del proyecto, así como otras variantes de SQL como PostgreSQL, priorizando la eficiencia en el procesamiento de consultas para este entorno específico.

Asimismo, se establecieron los flujos de interacción mediante diagramas que siguen el estándar **UML**, tomando como referencia los fundamentos de **Visual Paradigm (2023)**. El modelado técnico de estos diagramas se realizó mediante la herramienta

draw.io de **JGraph (s.f.)**, definiendo en primera instancia el diagrama de **Casos de Uso** (**ver Figura 2 en Anexo 1**). Para garantizar la robustez del sistema, cada caso de uso cuenta con su respectiva tabla de especificaciones, diagrama de actividad y diagrama de secuencia (**ver Anexo 2**), cubriendo las siguientes áreas funcionales:

- **Gestión de Accesos:** Inicio de sesión, registro, recuperación de contraseña y cierre de sesión **(ver Bloque A del Anexo 2).**
- **Gestión de Colección:** Búsqueda de títulos, adición a biblioteca personal y gestión de listas **(ver Bloque B del Anexo 2).**
- **Interacción Social:** Redacción de reseñas, sistema de valoraciones ("likes") y gestión de seguidores **(ver Bloque C del Anexo 2).**

Diseño Estético y Experiencia de Usuario: UX/UI

El diseño visual de la plataforma se fundamentó rigurosamente en los principios de las **Leyes de UX**, con el objetivo primordial de consolidar una interfaz intuitiva que minimice la carga cognitiva del usuario y realce los elementos de interacción clave. Para lograrlo, se aplicaron conceptos fundamentales como las **Leyes de Gestalt (ESDESIGN Barcelona, 2026)**, tales como la ley de proximidad y la de semejanza, que permiten organizar la información de manera que el cerebro humano la procese como un todo coherente y ordenado.

- **Identidad Visual:** Se definió una **paleta de colores** común (**ver Figura 1 en Anexo 3**) con tonos simples para resaltar los elementos visuales, como las carátulas de los juegos. Utilizando como referencia las plataformas como Steam, GOG o Humble Bundle, llegamos a la conclusión de que la mayoría de las plataformas deciden plasmar los productos en un entorno minimalista.
- **Prototipado:** Utilizando **Figma**, se crearon prototipos de alta fidelidad que permitieron validar el diseño y el flujo de navegación antes de la fase de programación (**ver Figuras 2 y 3 en Anexo 3**).

6.2. Desarrollo del Core: Backend y Base de Datos

Siguiendo nuestra planificación, se implementó el núcleo lógico del proyecto sobre un servidor web **Apache**. Para la administración y gestión visual de la base de datos,

se configuró la herramienta **phpMyAdmin** sobre dicho servidor, permitiendo una interacción directa con las tablas de **MySQL**. Como lenguaje de programación del lado del servidor se empleó **PHP 8.x (The PHP Group, 2024)**, aprovechando sus mejoras en rendimiento y tipado estricto para procesar la lógica de negocio de manera eficiente.

- **Persistencia de Datos:** Se construyó la base de datos en **MySQL**, cuya elección se fundamentó en su alta fiabilidad para garantizar la integridad referencial. Esta tecnología permitió estructurar eficientemente las relaciones necesarias para vincular los perfiles de usuario con el catálogo global de videojuegos, sus listas personalizadas y las interacciones sociales (reseñas y valoraciones).

- **API RESTful:** Se desarrollaron los *endpoints* necesarios para recibir peticiones HTTP y ejecutar operaciones **CRUD** (Crear, Leer, Actualizar, Borrar), devolviendo las respuestas en formato **JSON**. Esta API actúa como un puente unificado, permitiendo que tanto el cliente web como el móvil consuman y modifiquen la misma información en tiempo real de forma síncrona.

- **Sincronización y Volcado de Datos (IGDB):** Se diseñó un *script* especializado para la ingesta de información masiva, encargado de extraer los datos del catálogo desde la API de **IGDB (Twitch Interactive, s. f.-a)**. Para habilitar el acceso a estos *endpoints*, se integró el sistema de autenticación de la API de **Twitch (Twitch Interactive, s. f.-b)**. Se tomó la decisión estratégica de volcar y persistir estos datos en nuestra propia base de datos, en lugar de realizar llamadas directas desde los *endpoints* del cliente a la API externa. Esta medida se adoptó para mantener la fidelidad absoluta a nuestro modelo de datos relacional y asegurar que la integridad de las relaciones (como listas de usuarios vinculadas a IDs de juegos) no dependiera de la disponibilidad o cambios de estructura de servicios externos.

6.3. Desarrollo de Clientes: Web y Móvil

Una vez consolidada la plena operatividad del backend, se procedió al desarrollo concurrente de los entornos *front-end*, permitiendo que la plataforma web y la aplicación móvil evolucionaran de forma sincronizada. Este enfoque paralelo garantizó que ambas

interfaces compartieran una única fuente de datos, validando la versatilidad de la arquitectura distribuida para alimentar entornos heterogéneos de manera coherente. A continuación, se detallan las especificaciones técnicas y herramientas empleadas para la implementación de cada uno de estos entornos.

6.3.1. Cliente Web Front-end: React

Para el desarrollo de la plataforma accesible desde navegadores, se ha apostado por un ecosistema tecnológico orientado a la fluidez, la modularidad y la eficiencia en el manejo de datos masivos. La implementación técnica se ha centralizado en el uso de **Visual Studio Code (Microsoft, 2024)** como entorno de desarrollo principal. Tras evaluar alternativas profesionales como **WebStorm**, por su naturaleza de IDE integral, o **Zed**, por su alto rendimiento en arquitecturas nativas, se optó por Visual Studio Code debido a su ligereza, versatilidad y su ecosistema de extensiones especializado para el flujo de trabajo con React.

Dentro de este entorno, las tecnologías clave que vertebran el proyecto son:

- **React.js:** Se ha seleccionado esta biblioteca de JavaScript **(Meta Open Source, 2024)** para construir interfaces de usuario basadas en componentes reutilizables. Su uso frente a alternativas como Angular o Vue.js se justifica por su eficiente gestión del estado mediante el *Virtual DOM* y su flexibilidad con JavaScript nativo, lo que permitió una integración más ágil con los datos dinámicos sin la sobrecarga técnica de tipados estrictos como los de TypeScript.
- **Vite:** Como herramienta de construcción (*build tool*) y servidor de desarrollo, se ha empleado **Vite (2024)**. A diferencia de empaquetadores tradicionales como Webpack, que requieren procesar el grafo de dependencias completo antes del arranque, Vite utiliza *Native ESM* para ofrecer tiempos de carga casi instantáneos. Esta decisión técnica optimizó el flujo de trabajo al evitar las esperas de reconstrucción de paquetes habituales en proyectos de gran envergadura.
- **Gestión de Dependencias:** El control y la integración de las bibliotecas necesarias para el proyecto se han gestionado mediante **NPM (2023)**, asegurando un entorno de ejecución estable y una resolución de dependencias consistente durante todas las fases de desarrollo.

Conexión con el Backend y Enrutamiento:

La arquitectura funcional de la plataforma se completa con la integración de la lógica

de negocio a través del servidor. Para ello, se han implementado archivos de rutas específicos que gestionan las peticiones entre el cliente y el servidor. Este sistema permite:

1. **Procesamiento de peticiones:** Gestión de operaciones críticas como el inicio de sesión (*login*), registro de usuarios y recuperación de contraseñas mediante métodos HTTP (POST).
2. **Seguridad y Lógica:** El uso de estas rutas facilita la integración de procesos de cifrado (como *bcrypt*) y la interacción con el pool de conexiones de la base de datos, asegurando que cada acción del usuario en el frontend tenga una respuesta síncrona y segura en el backend.

El resultado es una **SPA (Single Page Application)** que ofrece una navegación *responsive* fluida por el catálogo sin recargas de página, optimizando la experiencia del usuario al gestionar grandes volúmenes de información **(ver Figuras 1 a 6 del Anexo 4).**

6.3.2. Aplicación Móvil Nativa: Android

Se ha creado una aplicación que aprovecha los recursos nativos del sistema mediante el uso de estas tecnologías:

- **Android Studio y Java:** Hemos utilizado el entorno de desarrollo integrado (IDE) oficial de Google, **Android Studio (Google Developers, 2024)**. Separando el desarrollo en dos pilares principales, el maquetado está escrito en **XML** al contrario que toda la parte lógica que está hecha en **Java (Oracle, 2023)**. Para conseguir esta última parte tuvimos que hacer downgrade de la versión ya que en las más modernas se programa en **Kotlin**, además de ser un lenguaje más familiar para el equipo de desarrollo.
- Android Studio tiene un sistema de emulación integrado utilizando SDK de dispositivos Android reales, gracias a esto pudimos acceder directamente a los recursos de hardware y a los componentes de interfaz del sistema operativo. Esto garantiza la correcta implementación del maquetado planteado al principio del desarrollo, las transiciones entre las distintas pantallas y la respuesta a los gestos táctiles que el usuario realiza (**ver Figuras de 7 a 14 del Anexo 4**).
- **Arquitectura de Red y Retrofit:** La comunicación con el backend se ha articulado mediante la biblioteca **Retrofit**, que actúa como un cliente HTTP de tipo *type-safe*. Esta tecnología permite transformar la API REST en una

interfaz de programación robusta, facilitando la gestión de peticiones de forma asíncrona.

- **Definición de Contratos:** La conexión se gestiona a través de interfaces como ApiAuthService, donde se definen declarativamente los *endpoints*, métodos HTTP (como POST o GET) y las estructuras de datos, desacoplando la definición de la API de su implementación técnica.
- **Gestión de Modelos y Serialización:** Se han implementado clases de modelos específicas para gestionar las solicitudes (*requests*) y respuestas (*responses*), encargadas de encapsular datos críticos como credenciales o tokens. Mediante conversores como GSON, el sistema automatiza la serialización y deserialización de objetos, transformando las respuestas JSON del servidor en objetos Java para su procesamiento inmediato en la lógica de la aplicación.

6.4. Integración, Pruebas y Sincronización

Durante las pruebas de integración, se llevaron a cabo sesiones de verificación ejecutando simultáneamente el cliente web y la aplicación Android (a través del simulador nativo) en la misma estación de trabajo. Este procedimiento permitió confirmar que cualquier modificación realizada en uno de los entornos se reflejaba de manera instantánea al actualizar la interfaz en el otro, independientemente de la plataforma de origen.

Esta sincronización en tiempo real ratifica que la API desarrollada en PHP y el servidor Apache operan correctamente como un puente unificado, cumpliendo con el objetivo estratégico de mantener la consistencia absoluta en la colección del usuario sin importar la arquitectura tecnológica o el dispositivo que se utilice.

6.5. Despliegue y Puesta en Producción

El despliegue de la infraestructura se abordó como un proceso iterativo de selección tecnológica, buscando la máxima disponibilidad y eficiencia. Inicialmente, se descartó el uso de **ngrok**, ya que, a pesar de permitir la exposición de servicios locales, mantenía una dependencia crítica de un dispositivo físico encendido, lo que vulneraba el objetivo de autonomía del sistema.

Posteriormente, se inició una migración hacia **Supabase**; tras adaptar el modelo

relacional a **PostgreSQL**, la opción fue desestimada debido a incompatibilidades directas entre la lógica del backend —basada en conexiones API propias— y el ecosistema cerrado de la plataforma. En una tercera etapa, se optó por **Google Cloud**, configurando la base de datos en **Cloud SQL**. Sin embargo, este enfoque presentó obstáculos insalvables debido a errores en el empaquetado de imágenes **Docker** al intentar migrar el backend a **Cloud Run**, sumado a la excesiva dependencia de **Cloud Jobs** para la ejecución de los *scripts* de volcado de datos desde la API de IGDB.

Tras evaluar y probar alternativas como **Aiven** y **Clever Cloud**, se optó finalmente por una arquitectura de servicios independientes:

- **Base de Datos:** Se realizó el despliegue en **Railway (s. f.)**, aprovechando su motor relacional con **MySQL** para asegurar la persistencia y la escalabilidad del esquema modelado. Es importante destacar que, debido a las condiciones del **plan gratuito** del proveedor, el servicio dispone de una velocidad de procesamiento de peticiones limitada y se mantiene operativo de forma estrictamente temporal bajo el periodo de prueba ofrecido. No obstante, esta configuración resulta suficiente para la validación del sistema en un entorno real de producción, demostrando que el software es plenamente funcional y quedando su rendimiento óptimo supeditado a la migración hacia una infraestructura con recursos garantizados.
- **Backend:** Se implementó como un servicio web en **Render (s.f.)**, configurando un entorno de ejecución continuo que permite la escucha de peticiones de forma distribuida, garantizando la conexión estable con la base de datos alojada en Railway. Cabe destacar que, aunque el software es plenamente funcional y operativamente completo, su despliegue actual se encuentra condicionado por las restricciones técnicas propias del **plan gratuito** de Render. Esta limitación implica que la infraestructura no está dimensionada para procesar una alta densidad de peticiones simultáneas; asimismo, el servicio entra en un estado de suspensión automática tras periodos de inactividad, lo que genera una latencia de reactivación y posibles errores de *timeout* en el primer acceso. No obstante, el sistema es técnica y lógicamente viable para un entorno profesional, condicionado únicamente a la migración hacia planes de pago que garanticen una disponibilidad constante.

Finalmente, el despliegue del **cliente web** se ejecutó mediante **GitHub Pages**. Este proceso se realizó a través de un flujo de integración continua que automatiza la compilación de los archivos **JSX** de React hacia activos estáticos de producción. El despliegue aprovecha la infraestructura de *edge computing* de GitHub para minimizar la latencia y asegurar que la interfaz sea accesible globalmente de forma segura a través de protocolos HTTPS.

7. Conclusiones y discusión.

Después de haber planteado, diseñado y ejecutado Memory Card, estas son las conclusiones principales que hemos llegado:

7.1. Aportación del proyecto

La mayor ventaja de este trabajo es la resolución de un problema real para los usuarios que poseen bibliotecas de juegos dispersas en múltiples plataformas digitales y físicas. Haber diseñado un sistema que unifica una aplicación móvil nativa y una plataforma web bajo un mismo "cerebro" (el backend) centralizado permite que el usuario mantenga el control de su colección en cualquier momento y lugar.

Al igual que plataformas como Letterboxd transformaron la forma de compartir y tener un registro del consumo cinematográfico, "Memory Card" aspira a profesionalizar el seguimiento del videojuego. El sistema no solo actúa como un inventario, sino como un diario personal y social donde el usuario puede registrar sus experiencias, valoraciones y reseñas.

7.2. Reflexión técnica: Discusión

Durante el proceso de desarrollo, se ha evidenciado que el reto principal de un sistema multiplataforma no reside en la programación aislada de cada cliente, sino en lograr una **interoperabilidad perfecta** y bidireccional entre ellos. La discusión técnica se centró en la arquitectura del sistema, la cual debía ser extremadamente sólida para garantizar que la información fuera consistente en todos los puntos de acceso y se transmitiera con la mínima latencia posible.

- **Desafíos de la Base de Datos:** Se dedicó un esfuerzo considerable a la normalización del esquema en **MySQL**. La discusión técnica giró en torno a garantizar la integridad referencial, asegurando que cualquier cambio de estado en la base de datos se propagara con total fidelidad, evitando cuellos de botella en la sincronización entre los distintos terminales.

- **Diseño de Interfaz de Usuario:** Finalmente nos decantamos por una interfaz limpia e intuitiva, utilizando paletas de colores simples y de la misma gama ayudando a resaltar las carátulas de los videojuegos y los elementos

importantes.

- **Sincronización y Backend:** La implementación de una **API RESTful** sobre **PHP 8.x** resultó ser la decisión arquitectónica más acertada. La flexibilidad de este entorno permitió procesar operaciones **CRUD** en tiempo real, actuando como un núcleo lógico capaz de servir datos estructurados en JSON de manera uniforme a tecnologías tan dispares como Java y React.

- **Cliente Web:** El uso de **React** junto a **Vite** permitió validar las ventajas de las **Single Page Applications (SPA)** frente al desarrollo web tradicional. La principal reflexión técnica en este apartado fue la eficiencia del *Virtual DOM* para renderizar grandes volúmenes de datos provenientes de la API de IGDB, logrando una interfaz altamente reactiva que responde de manera instantánea a las interacciones del usuario sin recargas de página.

- **Entorno Móvil:** El desarrollo nativo en Java para Android permitió comprender las limitaciones y ventajas de los recursos de hardware frente al entorno web. Independientemente de haber hecho *downgrade* a versiones antiguas del IDE, pudimos realizar todos los rasgos que planteamos. El uso de Android Studio y el maquetado en XML facilitaron una navegación fluida adaptada a gestos táctiles.

- **Despliegue y Puesta en Producción:** La transición hacia un entorno de producción real mediante **Render** y **Railway** aportó una valiosa perspectiva sobre la computación en la nube. Se concluyó que, si bien el software es plenamente funcional, las limitaciones de los planes gratuitos (como la latencia de reactivación o *spin down*) representan el único escollo para un rendimiento de grado profesional, validando no obstante la viabilidad lógica del sistema para una futura escalabilidad en planes de recursos garantizados.

7.3. Relación con lo aprendido en el Grado

Este TFG ha representado la culminación práctica de los conocimientos adquiridos en el ciclo de DAM. El proyecto pasó de ser la simple escritura de código para hacer una aplicación a finalmente ser un ejercicio integral de ingeniería de software que incluye:

- Gestión de control de versiones mediante **Git** y **GitHub.**
- Organización y metodologías ágiles (**SCRUM**) coordinadas en **Notion**.
- Diseño de UX y maquetado en **Figma**.
- Comprensión del flujo de datos distribuido desde el servidor hasta los dispositivos finales.
- Estar al frente de un desarrollo completo nos ha permitido a todo el equipo interiorizar cómo se relacionan los componentes de una aplicación profesional, desde las conexiones de la API hasta la implementación de bases de datos en el frontend, tanto de la interfaz web como de la aplicación móvil nativa, ofreciendo una visión real del entorno laboral en una empresa de desarrollo.

7.4. Trabajo futuro y líneas de mejora

Para garantizar la escalabilidad y la relevancia de **"Memory Card"** a largo plazo, se han identificado las siguientes líneas estratégicas de desarrollo que permitirían evolucionar el prototipo actual hacia un producto comercial de alto rendimiento:

- **Seguridad y Auditoría Avanzada:** Es prioritario someter la aplicación Android a análisis de seguridad exhaustivos (como pruebas de penetración y escaneo de vulnerabilidades en el manifiesto) para cumplir con los estándares de la Google Play Store. Asimismo, se plantea evolucionar el sistema de autenticación actual hacia el uso de **JSON Web Tokens (JWT)** o **OAuth 2.0**, mejorando la robustez en la transmisión de datos sensibles.
- **Adaptabilidad Web Multi-dispositivo:** Aunque la plataforma web actual es *responsive* y se ajusta con éxito a diversas resoluciones y formatos de monitor, una línea de mejora inmediata es optimizar el maquetado CSS para garantizar una compatibilidad total con pantallas de dispositivos móviles. Esto permitiría una experiencia de usuario fluida en navegadores de *smartphones*, complementando a la aplicación nativa.
- **Módulo de Analítica y Estadísticas:** Se proyecta el desarrollo de un panel de visualización de datos donde los usuarios puedan analizar métricas personales, como el tiempo de juego estimado o sus preferencias por

géneros. Esta funcionalidad aprovecharía la arquitectura relacional ya establecida en MySQL sin necesidad de reestructurar la base de datos. A su vez, añadir distintos tipos de conexiones con las diferentes plataformas para recibir notificaciones relacionadas con ofertas, lanzamientos o noticias relacionadas con los gustos de cada usuario.

- **Internacionalización:** Adaptar las interfaces de React y Android para soportar múltiples idiomas mediante el uso de librerías de internacionalización, expandiendo así el alcance social de la plataforma a una comunidad global.

- **Ecosistema iOS y Multiplataforma:** Dado que el sistema ya cuenta con clientes en React y Android Nativo, el siguiente paso lógico es el desarrollo de un cliente específico para **iOS** o la migración de los frontales hacia tecnologías híbridas para unificar el mantenimiento del código en todo el ecosistema móvil.

- **Persistencia Offline y Caché:** Implementar técnicas de almacenamiento local en el dispositivo móvil (como SQLite o Room) para permitir la consulta de la colección en modo *offline*, asegurando que los cambios se sincronicen automáticamente con el servidor una vez se recupere la conectividad.

8. Bibliografía.

- **SteamDB. (2026).** *Steam Game Release Statistics: Annual Summaries 2020-2025*. Recuperado de https://steamdb.info/stats/releases/
- **Yablonski, J. (s.f.).** *Leyes de UX*. Recuperado el 13 de marzo de 2026, de https://lawsofux.com/es/
- **Sequeiros Bruna, H., & Fernández Ruiz, M. (2022).** Persuasión en los videojuegos: La construcción de identidad. *Teknokultura: Revista de Cultura Digital y Movimientos Sociales*, *19*(1), 33-44. Recuperado de https://dialnet.unirioja.es/servlet/articulo?codigo=8257813
- **GitHub. (s.f.).** *GitHub Documentation*. GitHub Docs. Recuperado el [Día] de [Mes] de 2026, de https://docs.github.com/en/
- **Schwaber, K., & Sutherland, J. (2020).** La Guía de Scrum. Scrum.org. Recuperado de https://scrumguides.org/
- **MySQL. (2023).** MySQL 8.0 Reference Manual. Oracle Corporation. Recuperado de https://dev.mysql.com/doc/refman/8.0/en/
- **Visual Paradigm. (2023).** *Guía de aprendizaje de UML*. Visual Paradigm. https://www.visual-paradigm.com/es/guide/uml-unified-modeling-language/
- **JGraph. (s.f.).** *UML diagrams in draw.io*. diagrams.net. Recuperado el 13 de marzo de 2026, de https://www.diagrams.net/blog/uml-diagrams
- **ESDESIGN Barcelona. (2026, 2 de febrero).** *Teoría de la Gestalt: leyes clave aplicadas al diseño visual*. Recuperado de https://www.esdesignbarcelona.com/actualidad/diseno-grafico/teoria-gestalt
- **The PHP Group. (2024).** *PHP 8.x Manual*. PHP.net. Recuperado de https://www.php.net/manual/es/index.php
- **Twitch Interactive. (s. f.).** *IGDB API Reference*. IGDB API Docs. Recuperado el 1 de abril de 2026, de https://api-docs.igdb.com/#endpoints
- **Twitch Interactive. (s. f.).** *Twitch API Reference*. Twitch Developers. Recuperado el 1 de abril de 2026, de https://dev.twitch.tv/docs/api/
- **Microsoft. (2024).** *Visual Studio Code Documentation*. Recuperado de https://code.visualstudio.com/docs
- **Meta Open Source. (2024).** React Documentation: Describing the UI. Recuperado de https://react.dev/learn
- **Vite. (2024).** Next Generation Frontend Tooling. Recuperado de

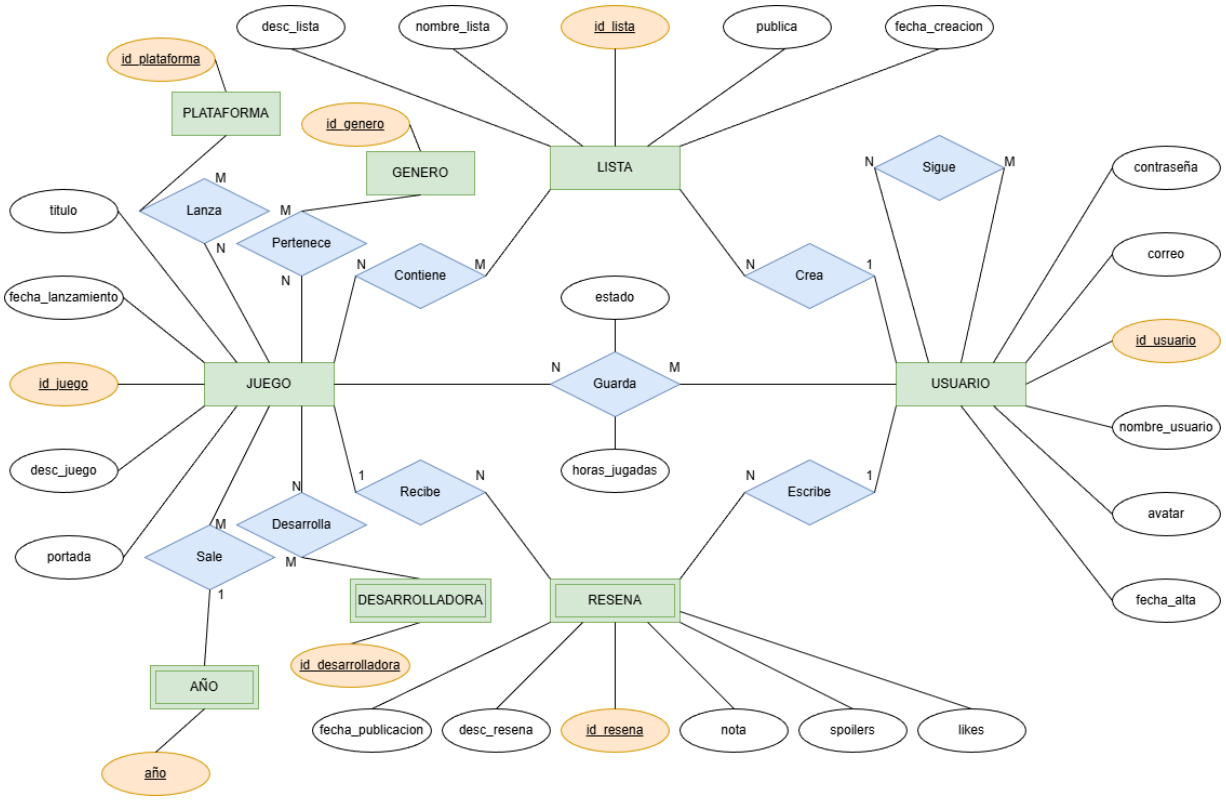
https://vitejs.dev/guide/

- **NPM. (2023).** *About npm*. npm Docs. Recuperado de https://docs.npmjs.com/
- **Google Developers. (2024).** *Android Studio User Guide*. Android Developers. Recuperado de https://developer.android.com/studio/intro
- **Oracle. (2023).** *Java Platform, Standard Edition Documentation (Release 21)*. Oracle Help Center. Recuperado de https://docs.oracle.com/en/java/javase/21/
- **Railway. (s.f.).** *Railway Documentation*. Recuperado el 10 de abril de 2026, de https://docs.railway.com/
- **Render. (s.f.).** *Render Documentation*. Recuperado el 11 de abril de 2026, de https://render.com/docs

9. Anexos

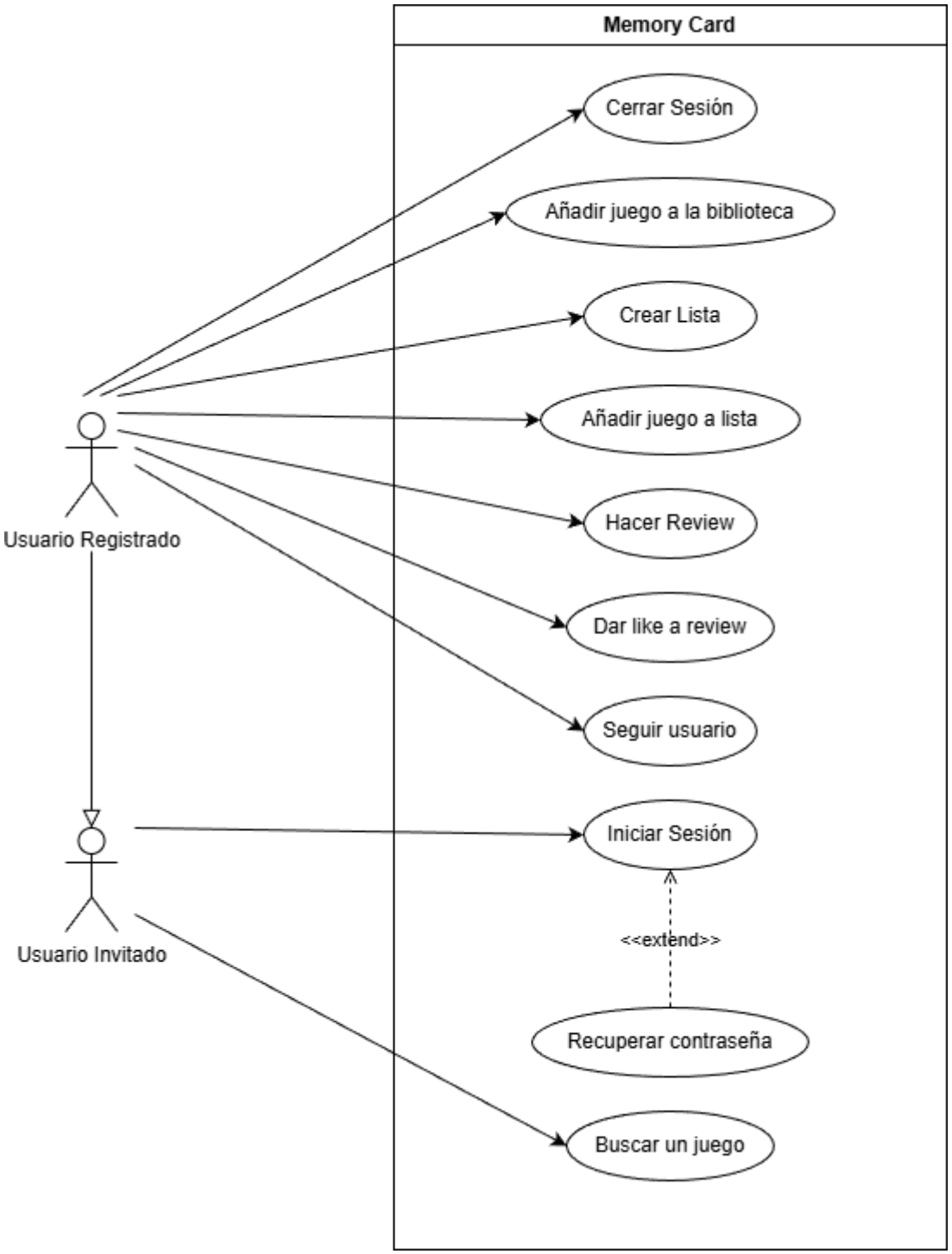
Anexo 1. Arquitectura de Datos y Funcional

- **Figura 1.** Diagrama Entidad-Relación de la Base de Datos (MySQL).



Descripción: Representación del modelo relacional que soporta la persistencia de usuarios, videojuegos y relaciones sociales.

- **Figura 2.** Mapa General de Casos de Uso (UML).



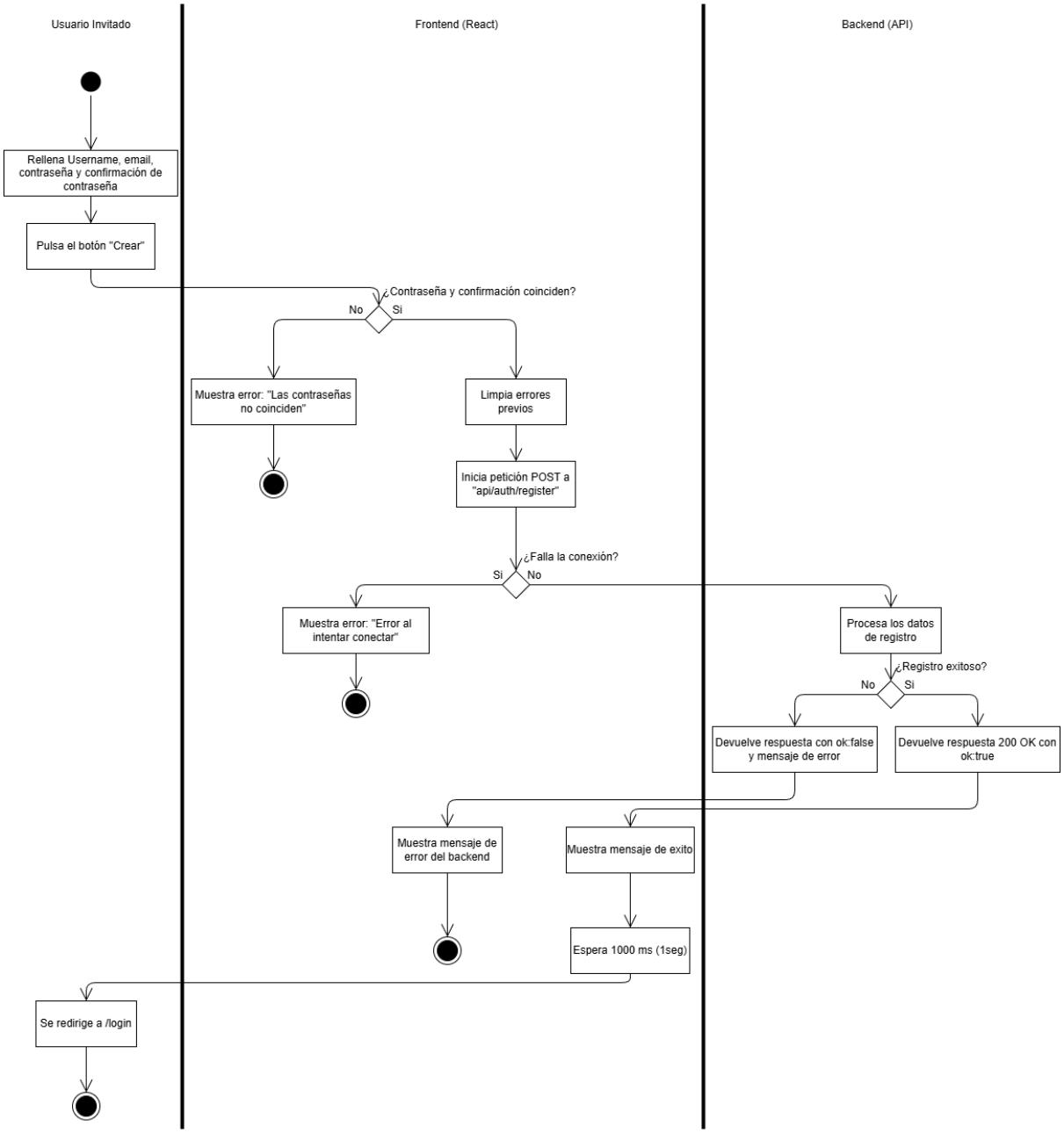
Descripción: Visión global de las principales funcionalidades y actores que interactúan con el ecosistema Memory Card.

Anexo 2. Especificación Técnica de Funcionalidades

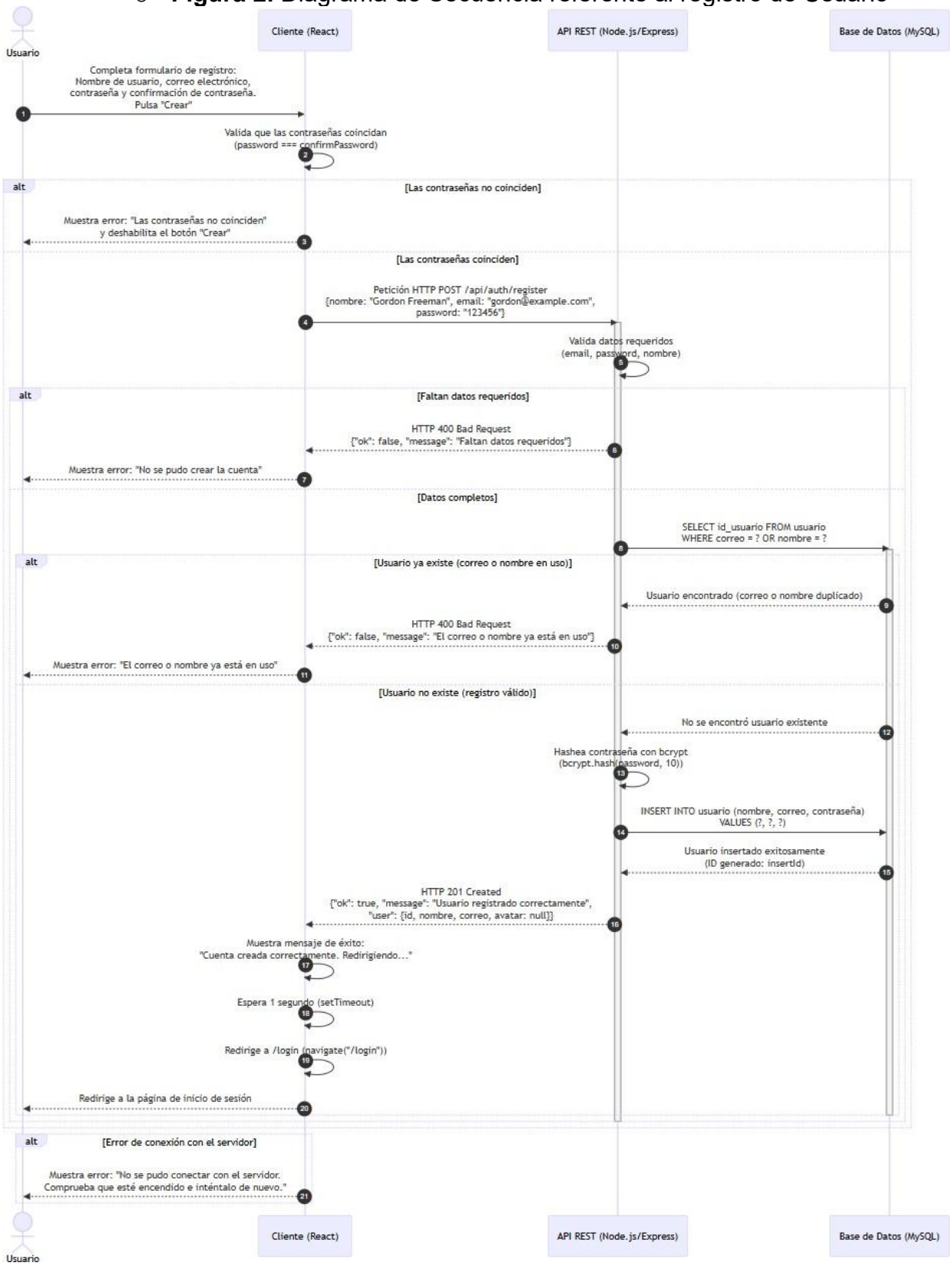
Este anexo contiene las **Tablas de Especificación**, los **Diagramas de Actividad** y los **Diagramas de Secuencia** para cada uno de los 11 casos de uso definidos.

Bloque A: Gestión de Accesos

- **Figuras 1 y 2 y Tabla 1.** Registro de usuario.
    - **Figura 1.** Diagrama de Actividad referente al registro de Usuarios.



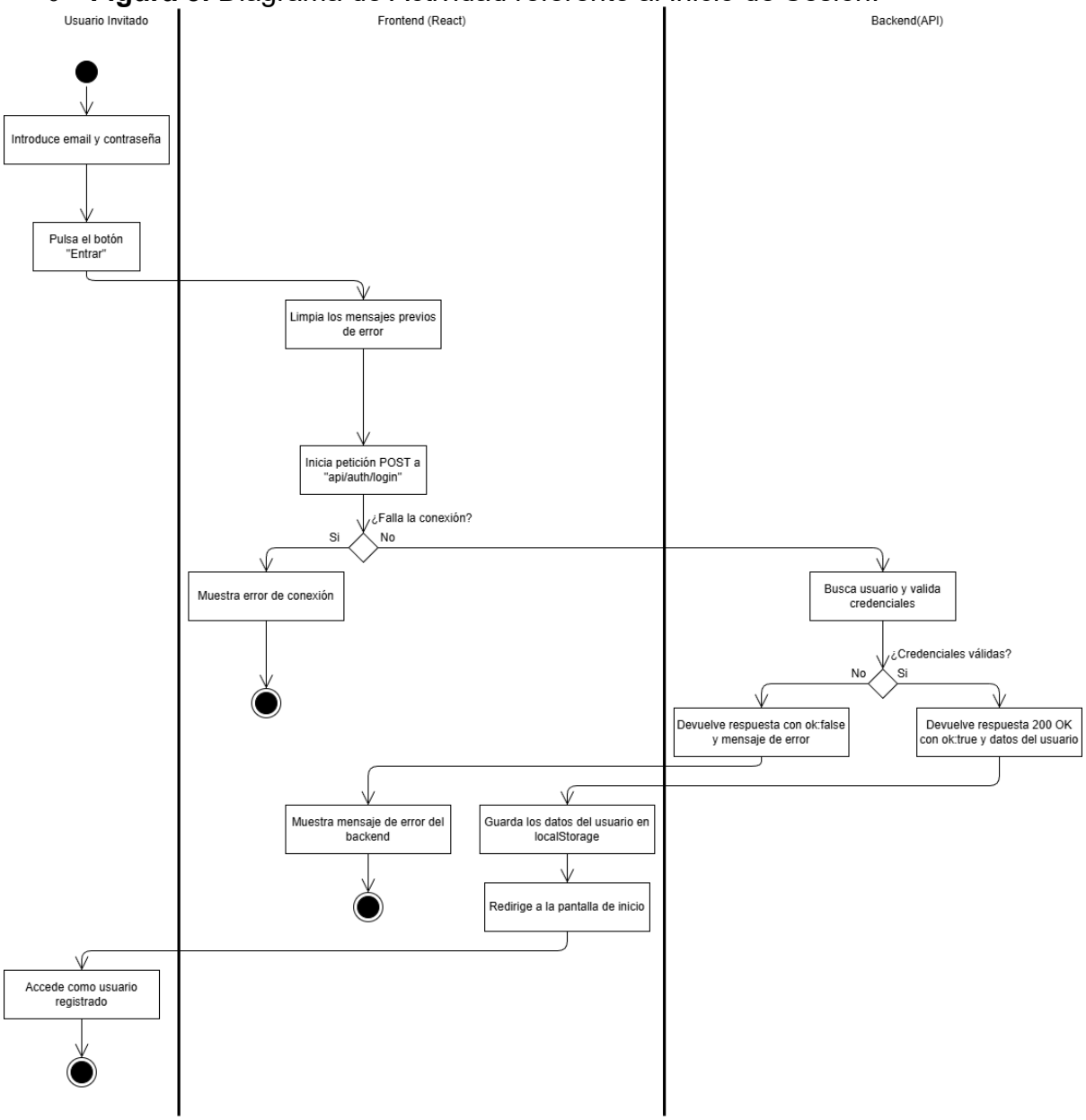
- **Figura 2.** Diagrama de Secuencia referente al registro de Usuario



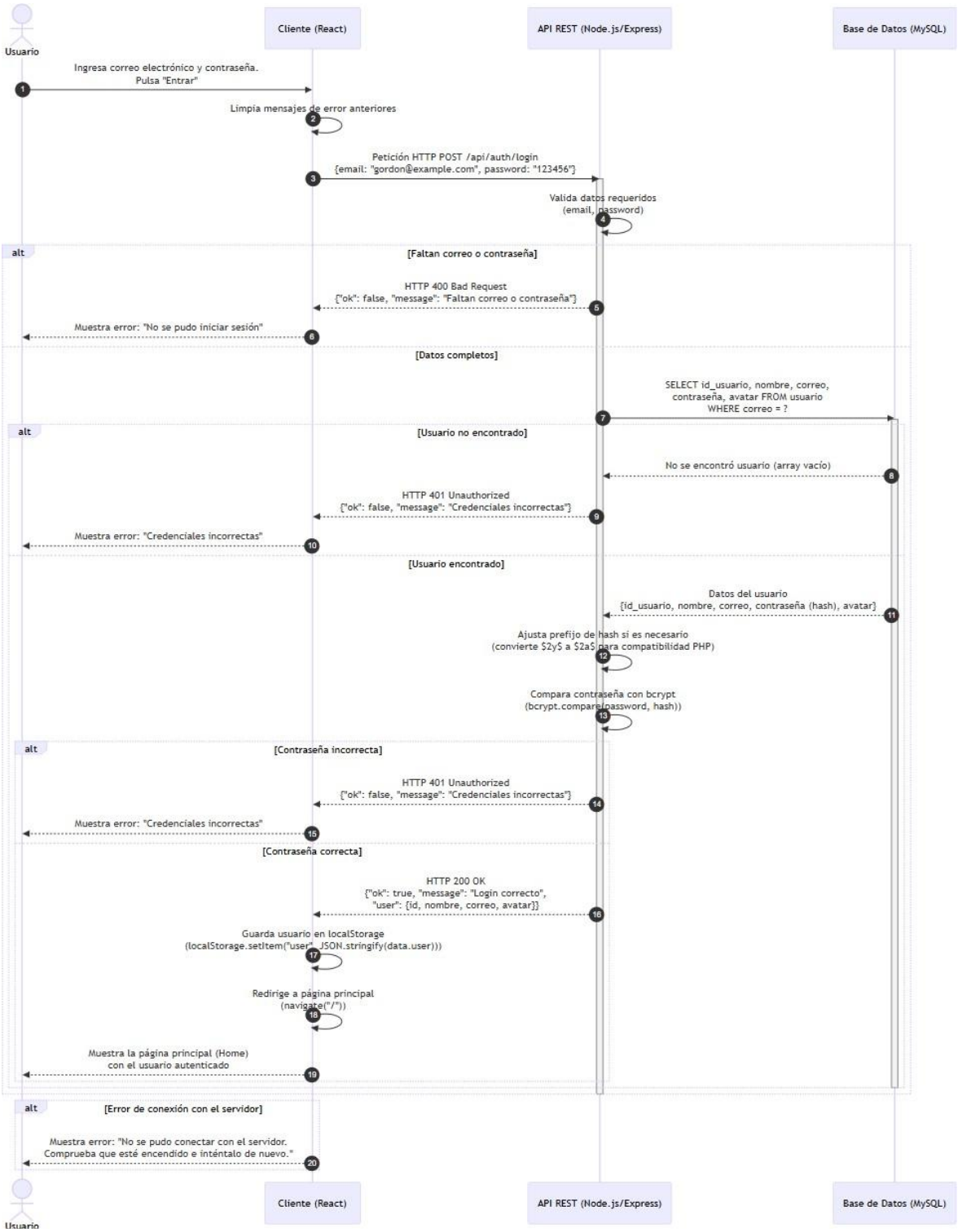
- **Tabla 1.** Tabla de especificación de requisitos referente al registro de Usuarios.

NOMBRE DEL CASO	Registrar nuevo usuario
DESCRIPCION	Un usuario no registrado previamente rellena el formulario para crear una cuenta
ACTORES	Usuario Invitado
PRECONDICIONES	El usuario no debe tener una cuenta creada ni una sesión iniciada
SECUENCIA NORMAL	1. El usuario introduce nombre de usuario, correo, contraseña y confirmación de contraseña. 2. El usuario pulsa el botón "Crear". 3. El sistema valida localmente que ambas contraseñas son idénticas. 4. El sistema envía los datos al servidor. 5. El servidor registra al usuario con éxito. 6. El sistema muestra un mensaje de éxito ("Cuenta creada correctamente"). 7. Tras 1 segundo, el sistema redirige al usuario a la pantalla de Inicio de Sesión.
POSTCONDICIONES	El usuario queda registrado en la base de datos
EXCEPCIONES	- **Contraseñas distintas:** El botón se deshabilita y/o el sistema muestra el error *"Las contraseñas no coinciden"*. No se llama al servidor. - **Fallo del servidor/Datos inválidos:** Si la API devuelve un error (ej. email ya en uso), se muestra el mensaje devuelto por el servidor o *"No se pudo crear la cuenta"*. - **Caída de red:** Si falla la petición fetch, se muestra *"Error de red al intentar conectar con el servidor"*.

- **Figuras 3 y 4 y Tabla 2.** Inicio de sesión.
  - **Figura 3.** Diagrama de Actividad referente al Inicio de Sesión.



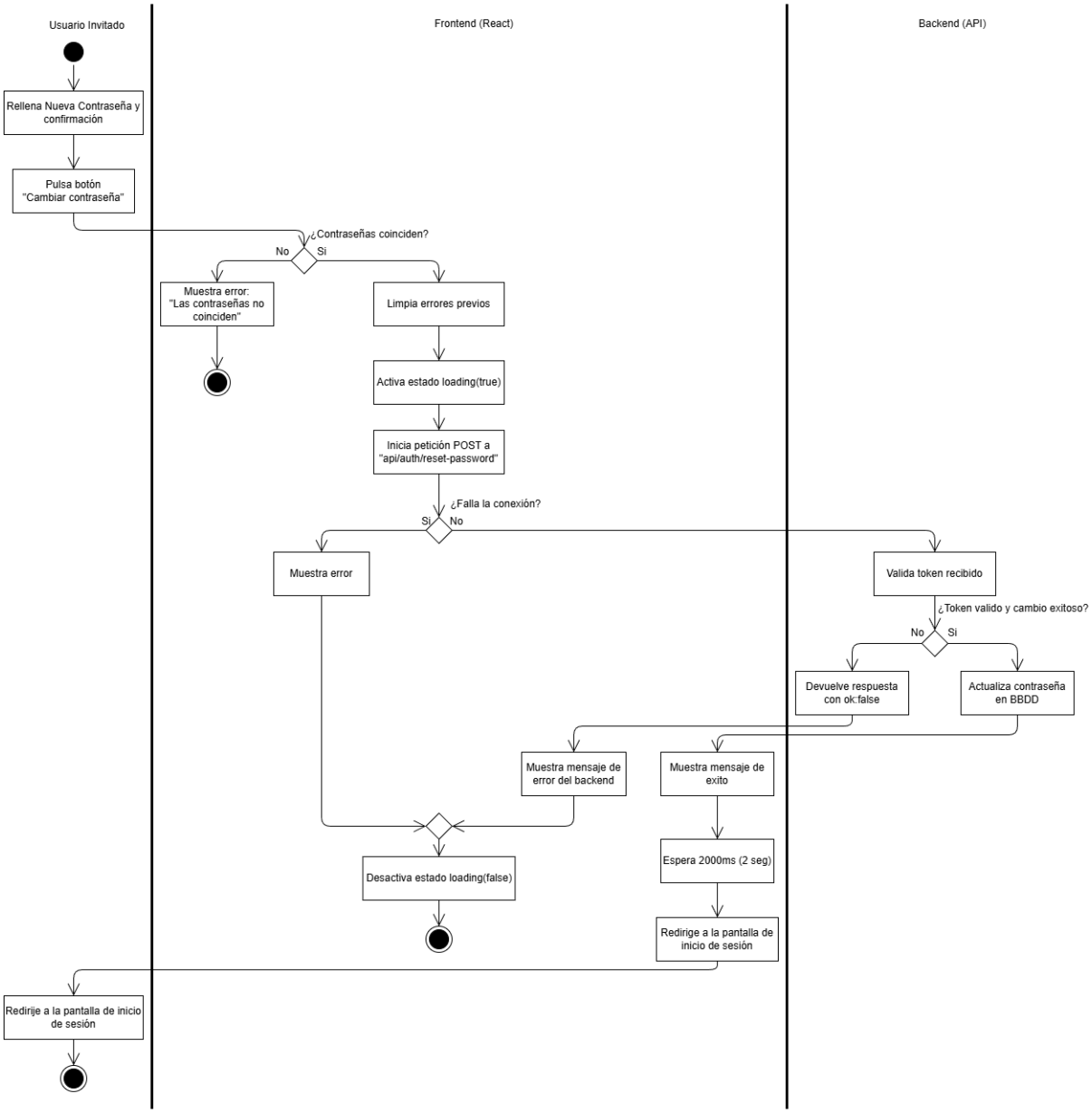
- **Figura 4.** Diagrama de Secuencia referente al Inicio de Sesión.



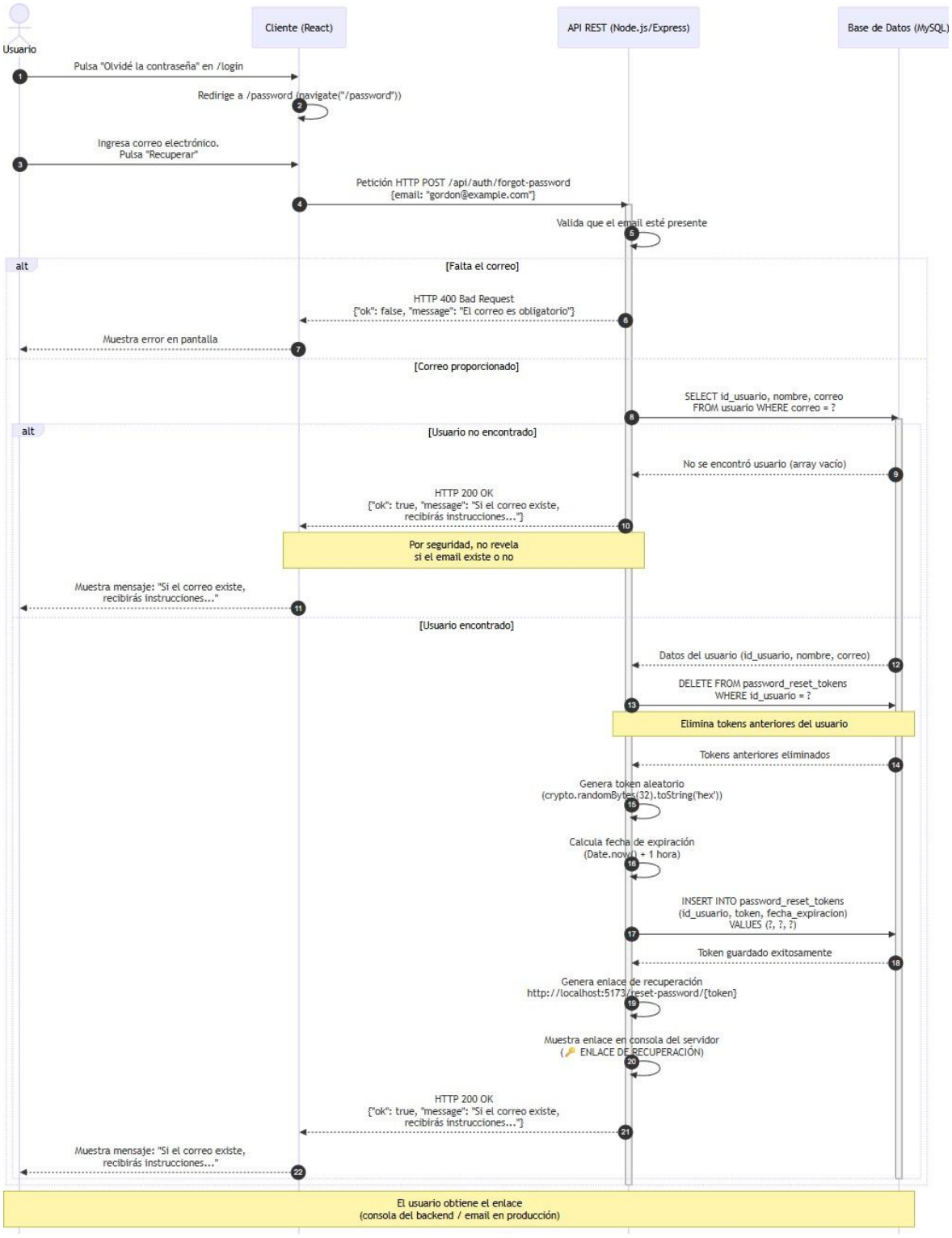
- **Tabla 2.** Tabla de especificación de requisitos referente al Inicio de Sesión.

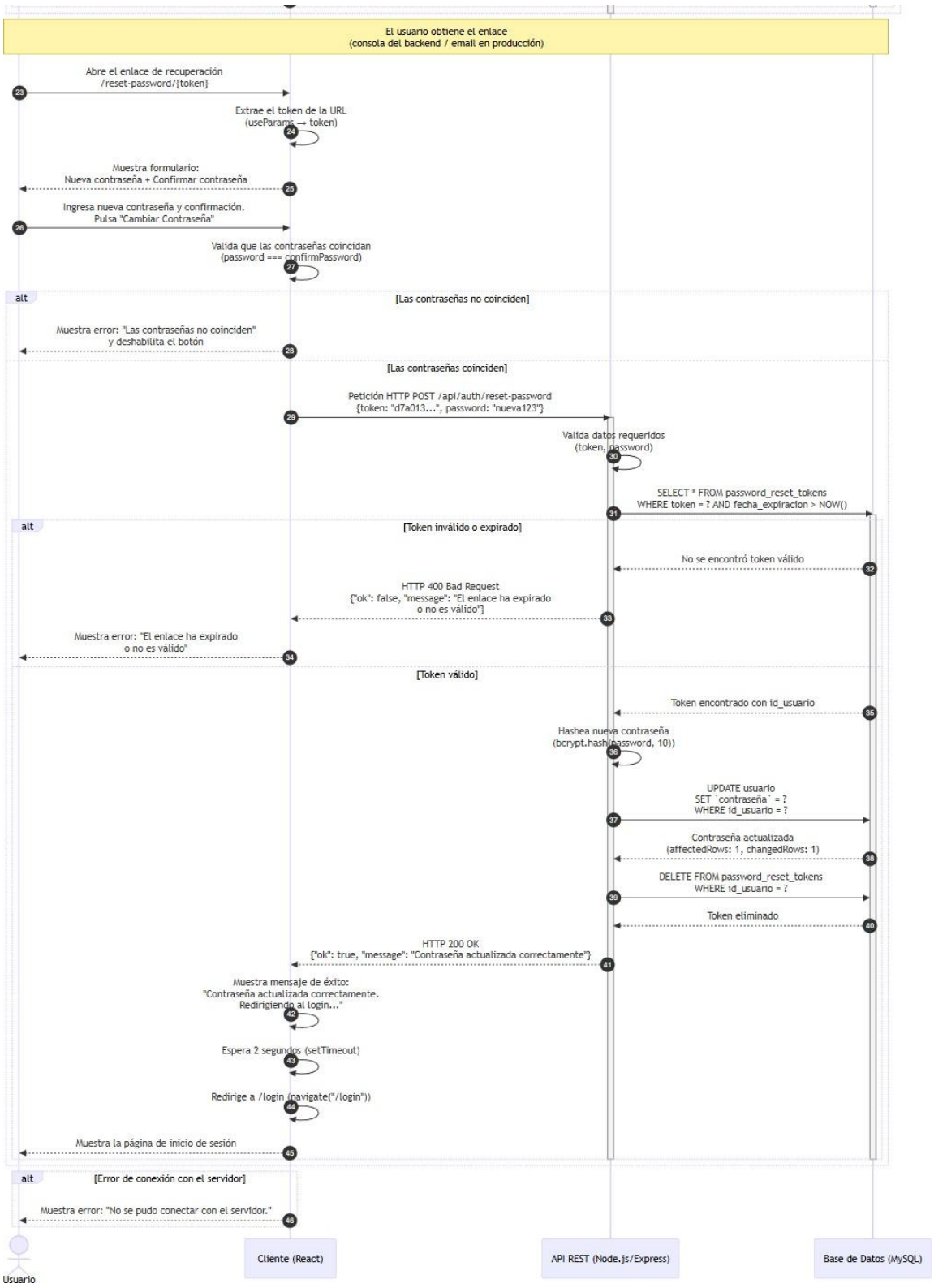
NOMBRE DEL CASO	Iniciar sesión
DESCRIPCION	Un usuario ya registrado accede a su cuenta rellenado el formulario
ACTORES	Usuario Invitado
PRECONDICIONES	El usuario debe estar registrado en la BBDD y no tener activa la sesión
SECUENCIA NORMAL	1. El usuario introduce su correo y contraseña. 2. El usuario pulsa el botón "Entrar". 3. El sistema limpia errores previos e inicia la petición al servidor. 4. El servidor valida las credenciales en la base de datos. 5. El servidor devuelve los datos del usuario. 6. El sistema guarda los datos del usuario en el almacenamiento local. 7. El sistema redirige al usuario a la página de Inicio.
POSTCONDICIONES	El usuario tiene una sesión activa y se encuentra en la pantalla principal.
EXCEPCIONES	- **Credenciales incorrectas:** El servidor devuelve un error. El sistema muestra el mensaje de error del backend o *"No se pudo iniciar sesión"*. No se guarda nada en el almacenamiento. **Caída de red:** Si la conexión falla (entra al bloque catch), se muestra *"Error de red al intentar conectar con el servidor"*.

- **Figuras 5 y 6 y Tabla 3.** Recuperación de contraseña.
  - **Figura 5.** Diagrama de Actividad referente a la recuperación de contraseña.



- **Figura 6.** Diagrama de Secuencia referente a la recuperación de contraseña.

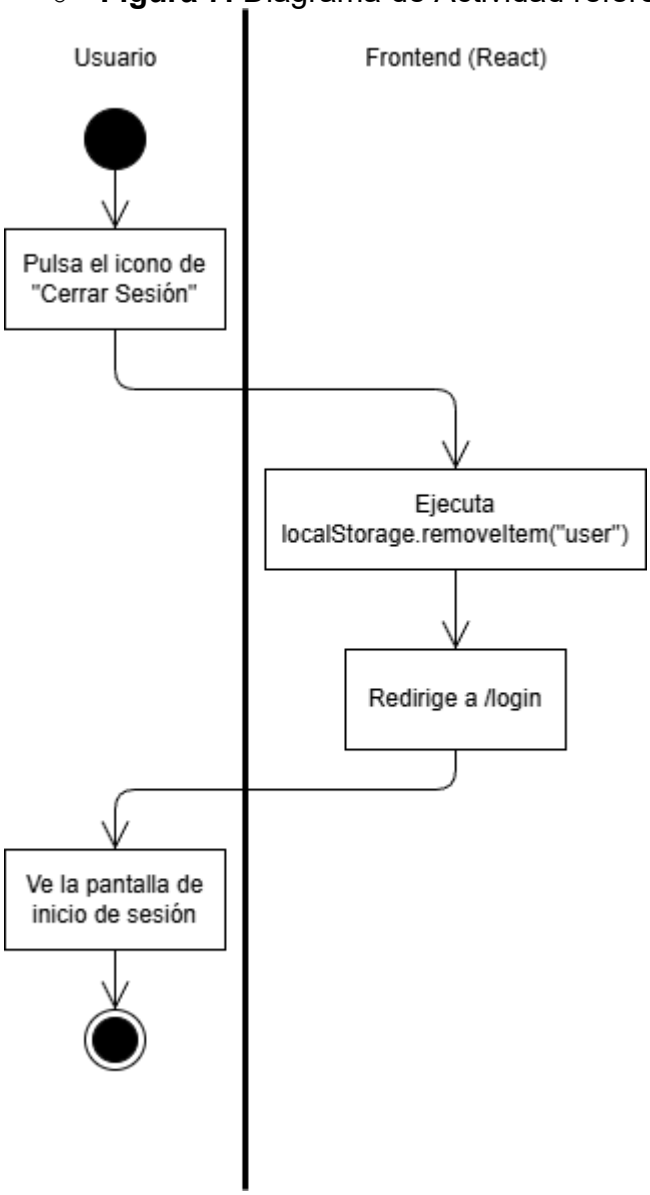



El usuario obtiene el enlace
(consola del backend / email en producción)
Abre el enlace de recuperación
/reset-password/{token}
Extrae el token de la URL
(useParams → token)
Muestra formulario:
Nueva contraseña + Confirmar contraseña
Ingresa nueva contraseña y confirmación.
Pulsa "Cambiar Contraseña"
Valida que las contraseñas coincidan
(password === confirmPassword)
alt
[Las contraseñas no coinciden]
Muestra error: "Las contraseñas no coinciden"
y deshabilita el botón
[Las contraseñas coinciden]
Petición HTTP POST /api/auth/reset-password
{token: "d7a013...", password: "nueva123"}
Valida datos requeridos
(token, password)
SELECT * FROM password_reset_tokens
WHERE token = ? AND fecha_expiracion > NOW()
alt
[Token inválido o expirado]
No se encontró token válido
HTTP 400 Bad Request
{"ok": false, "message": "El enlace ha expirado
o no es válido"}
Muestra error: "El enlace ha expirado
o no es válido"
[Token válido]
Token encontrado con id_usuario
Hashea nueva contraseña
(bcrypt.hash(password, 10))
UPDATE usuario
SET `contraseña` = ?
WHERE id_usuario = ?
Contraseña actualizada
(affectedRows: 1, changedRows: 1)
DELETE FROM password_reset_tokens
WHERE id_usuario = ?
Token eliminado
HTTP 200 OK
{"ok": true, "message": "Contraseña actualizada correctamente"}
Muestra mensaje de éxito:
"Contraseña actualizada correctamente.
Redirigiendo al login..."
Espera 2 segundos (setTimeout)
Redirige a /login (navigate("/login"))
Muestra la página de inicio de sesión
alt
[Error de conexión con el servidor]
Muestra error: "No se pudo conectar con el servidor."
Usuario
Cliente (React)
API REST (Node.js/Express)
Base de Datos (MySQL)

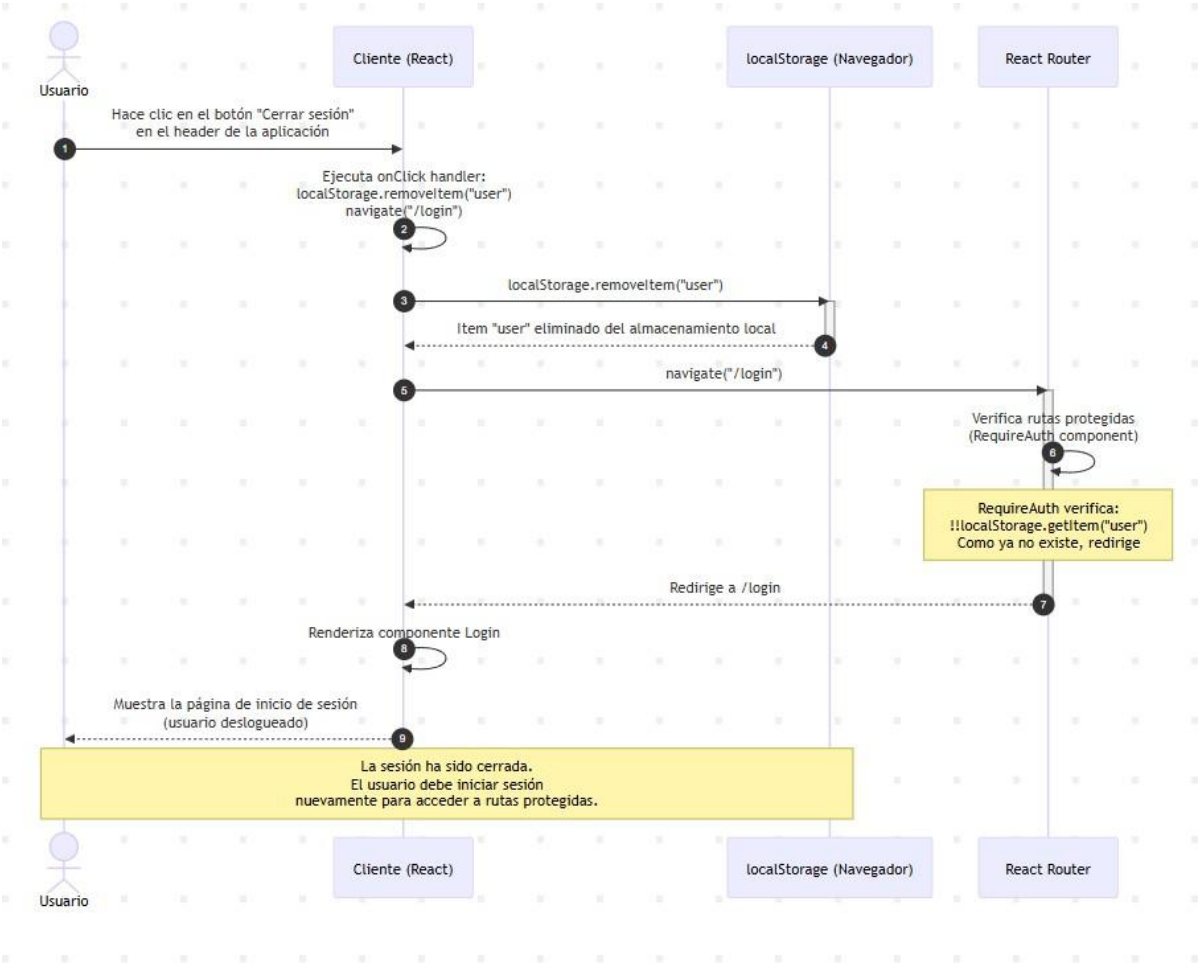
- **Tabla 3.** Tabla de especificación de requisitos referente a la recuperación de contraseña.

NOMBRE DEL CASO	Establecer nueva contraseña
DESCRIPCION	Un usuario que ha olvidado su contraseña introduce una nueva
ACTORES	Usuario invitado
PRECONDICIONES	El usuario debe haber accedido a la vista mediante un enlace válido que contenga un token de recuperación
SECUENCIA NORMAL	1. El usuario introduce su nueva contraseña y la confirmación. 2. El usuario pulsa el botón "Cambiar Contraseña". 3. El sistema verifica localmente que ambas contraseñas coinciden. 4. El sistema activa el estado de carga y envía el *token* y la contraseña al servidor. 5. El servidor valida el *token* y actualiza la contraseña en la base de datos. 6. El servidor devuelve una respuesta de éxito. 7. El sistema desactiva la carga, muestra un mensaje de éxito y, tras 2 segundos, redirige al Login.
POSTCONDICIONES	La contraseña del usuario se actualiza en la BBDD
EXCEPCIONES	- **Contraseñas distintas:** El botón se deshabilita y se muestra *"Las contraseñas no coinciden"*. No hay petición al servidor. - **Token inválido/caducado o error de backend:** El servidor rechaza la petición. Se muestra el mensaje del backend o *"No se pudo cambiar la contraseña"*. El estado de carga se desactiva. - **Caída de red:** Falla la conexión (catch). Se muestra el mensaje *"No se pudo conectar con el servidor..."* y el estado de carga se desactiva en el bloque finally.

- **Figuras 7 y 8 y Tabla 4.** Cierre de sesión.
  - **Figura 7.** Diagrama de Actividad referente al Cierre de Sesión.



- **Figura 8.** Diagrama de Secuencia referente al Cierre de Sesión.

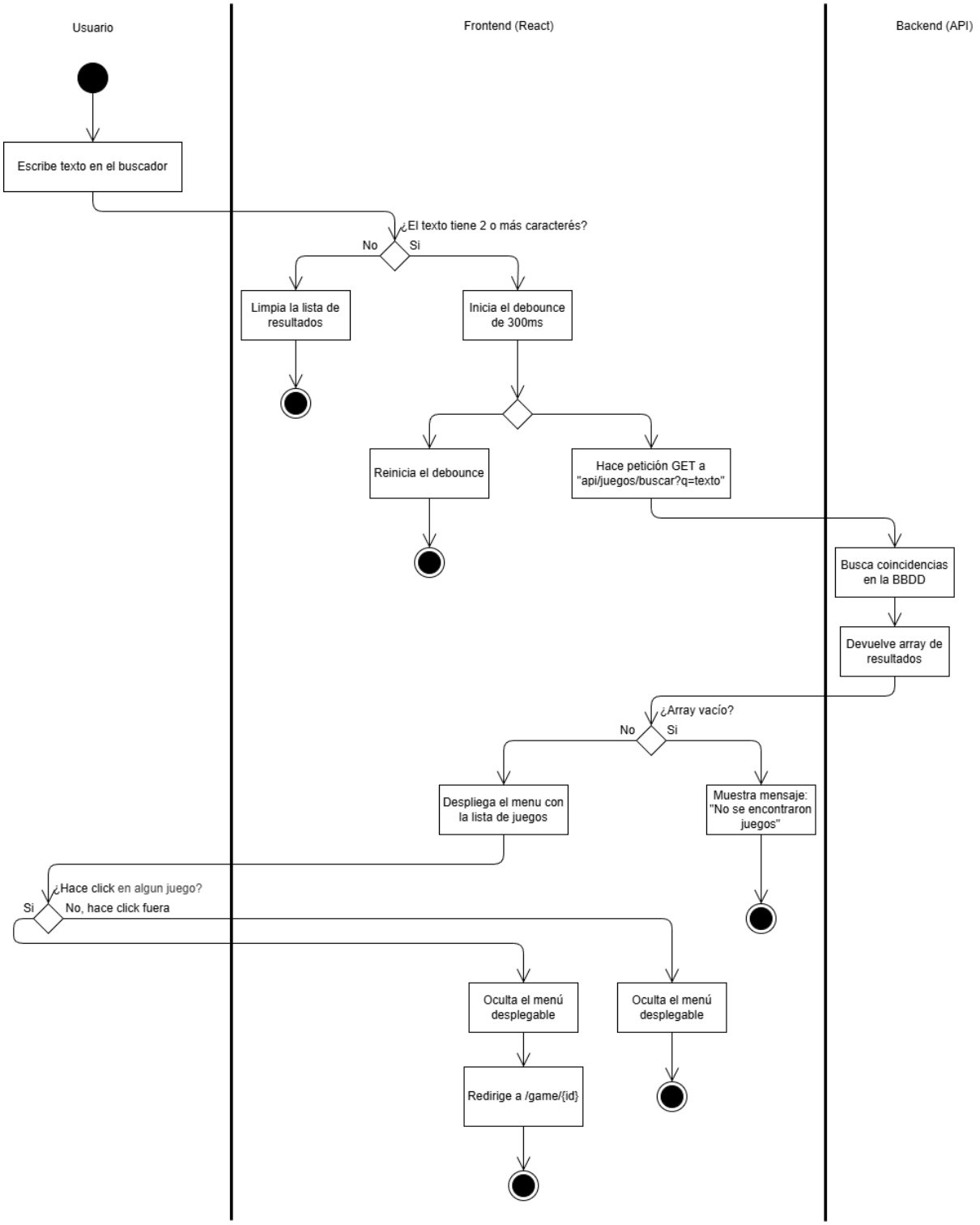


- **Tabla 4.** Tabla de especificación de requisitos referente al Cierre de Sesión.

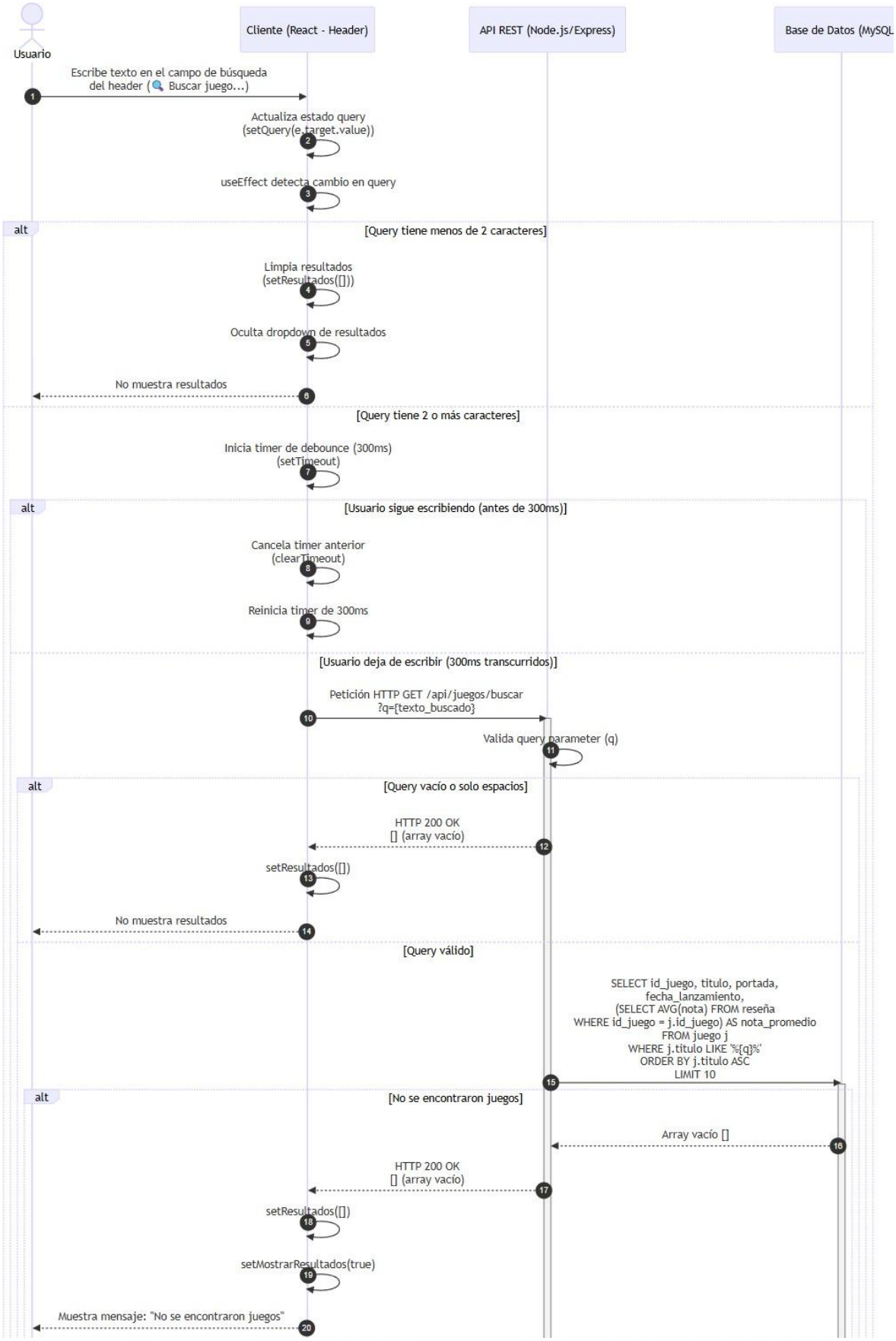
NOMBRE DEL CASO	Cerrar Sesión
DESCRIPCION	El usuario finaliza su sesión pulsando un botón
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe tener activa la sesión y estar en una pantalla que contenga el encabezado
SECUENCIA NORMAL	1. El usuario hace clic en el icono de "Cerrar sesión". 2. 2. El sistema elimina los datos de sesión del almacenamiento local del navegador. 3. 3. El sistema redirige al usuario a la pantalla de Inicio de Sesión.
POSTCONDICIONES	La sesión queda cerrada y el usuario no puede realizar ciertas acciones en el sistema.
EXCEPCIONES	Ninguna

Bloque B: Gestión de la Colección

- **Figuras 9 y 10 y Tabla 5.** Buscar Juego.
  - **Figura 9.** Diagrama de Actividad referente a Buscar Juego.



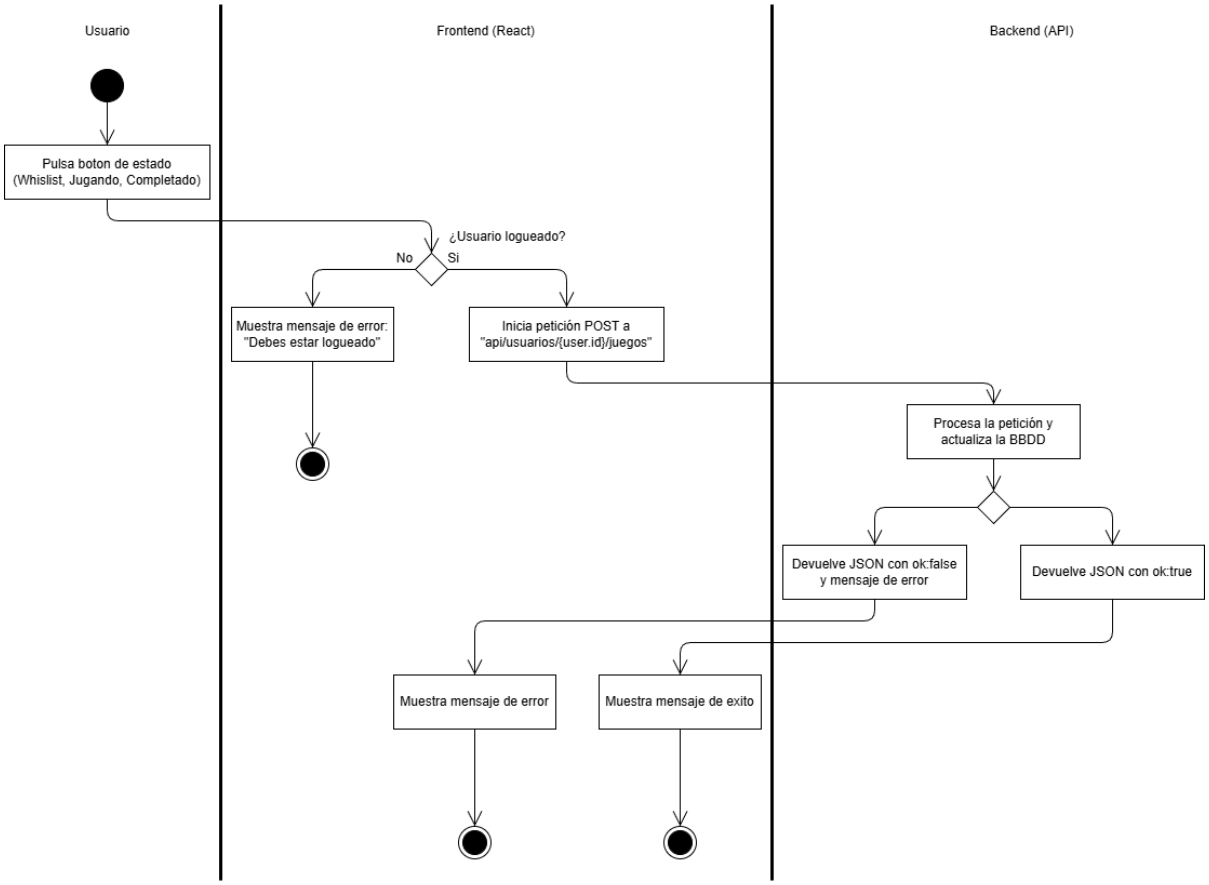
- **Figura 10.** Diagrama de Secuencia referente a Buscar Juego.



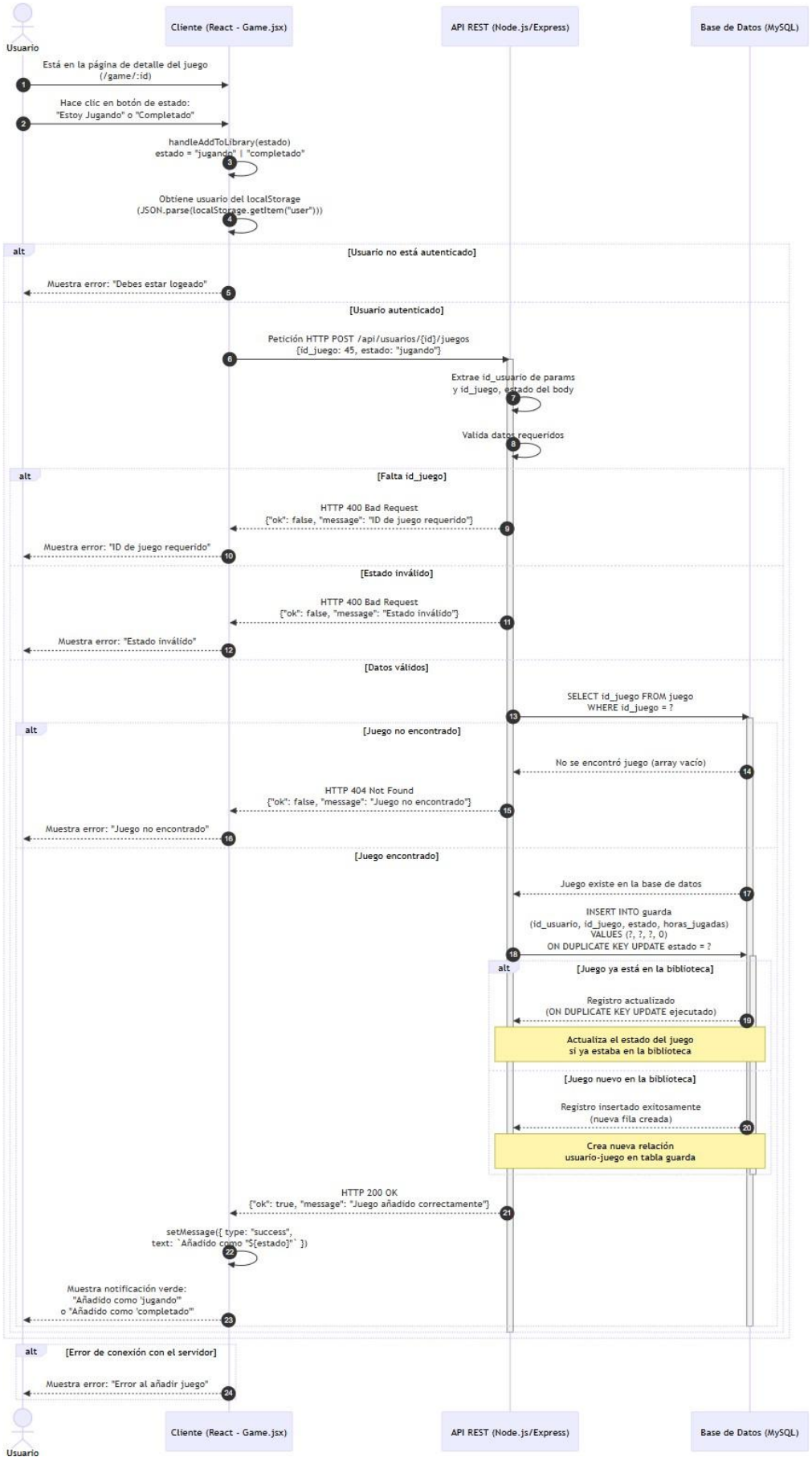
- **Tabla 5.** Tabla de especificación de requisitos referente al Buscar Juego.

NOMBRE DEL CASO	Buscar Juego
DESCRIPCION	El usuario busca un juego en la barra de búsqueda mientras que el sistema muestra los posibles resultados en tiempo real.
ACTORES	Usuario registrado
PRECONDICIONES	El usuario debe estar en una pantalla que contenga el encabezado
SECUENCIA NORMAL	1. El usuario teclea el nombre de un juego (mínimo 2 caracteres). 2. El sistema espera 300ms sin nueva entrada. 3. El sistema realiza una petición al servidor con el texto de búsqueda. 4. El servidor devuelve una lista de coincidencias. 5. El sistema despliega una lista con las miniaturas, títulos y valoraciones. 6. El usuario hace clic en un juego de la lista. 7. El sistema limpia el buscador y redirige a la ficha del juego.
POSTCONDICIONES	El usuario se encuentra en la página con los detalles del juego seleccionado
EXCEPCIONES	- **Menos de 2 caracteres:** El sistema vacía la lista localmente y no hace petición al servidor. - **Sin resultados:** El servidor devuelve un array vacío. El sistema muestra *"No se encontraron juegos para [texto]"*. - **Clic fuera del buscador:** El sistema oculta el menú desplegable sin borrar el texto. - **Error de red/API:** El error se captura silenciosamente en consola y no se muestran resultados.

- **Figuras 11 y 12 y Tabla 6.** Añadir juego a biblioteca.
  - **Figura 11.** Diagrama de Actividad referente a Añadir juego a biblioteca.



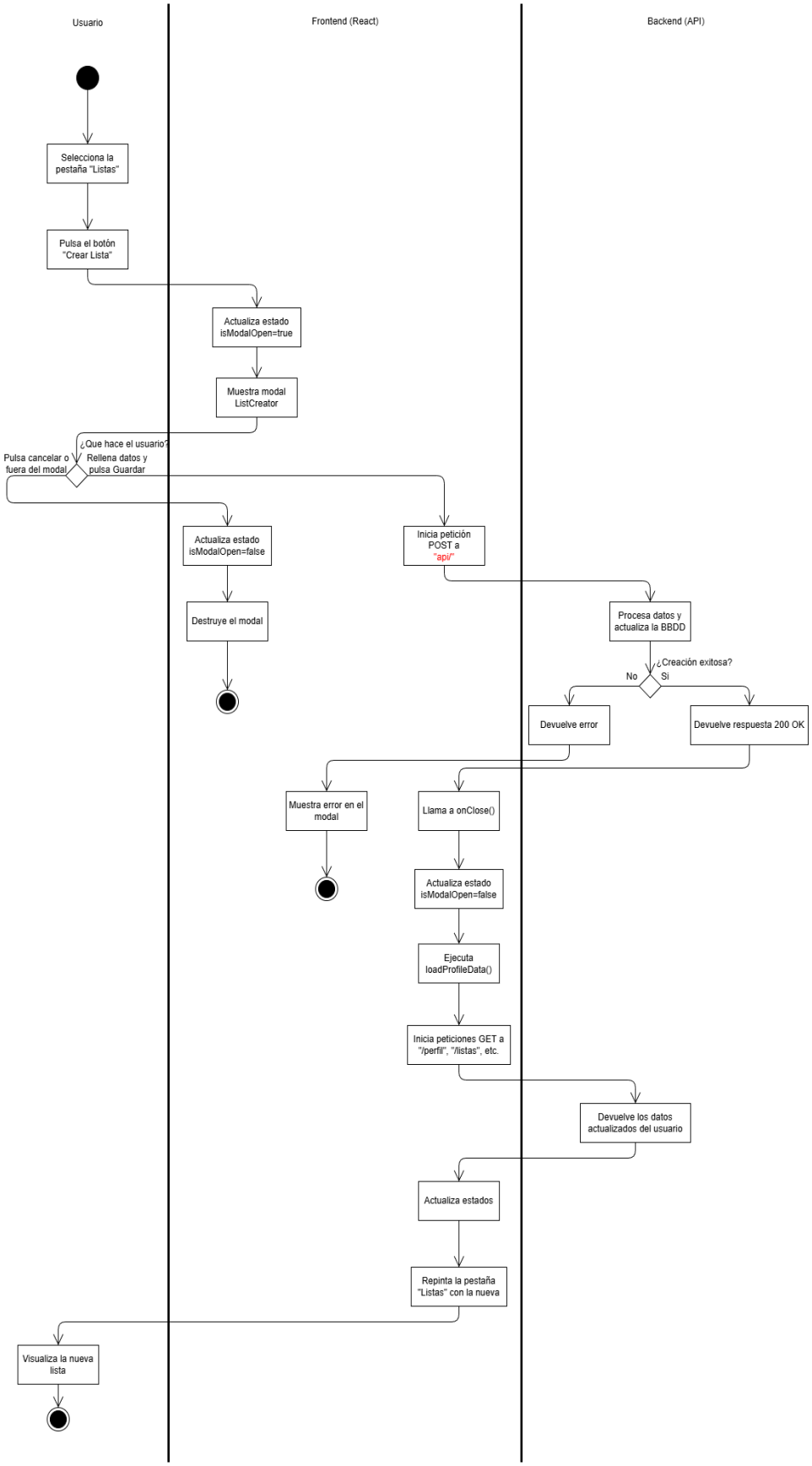
- **Figura 12.** Diagrama de Secuencia referente a Añadir juego a biblioteca.



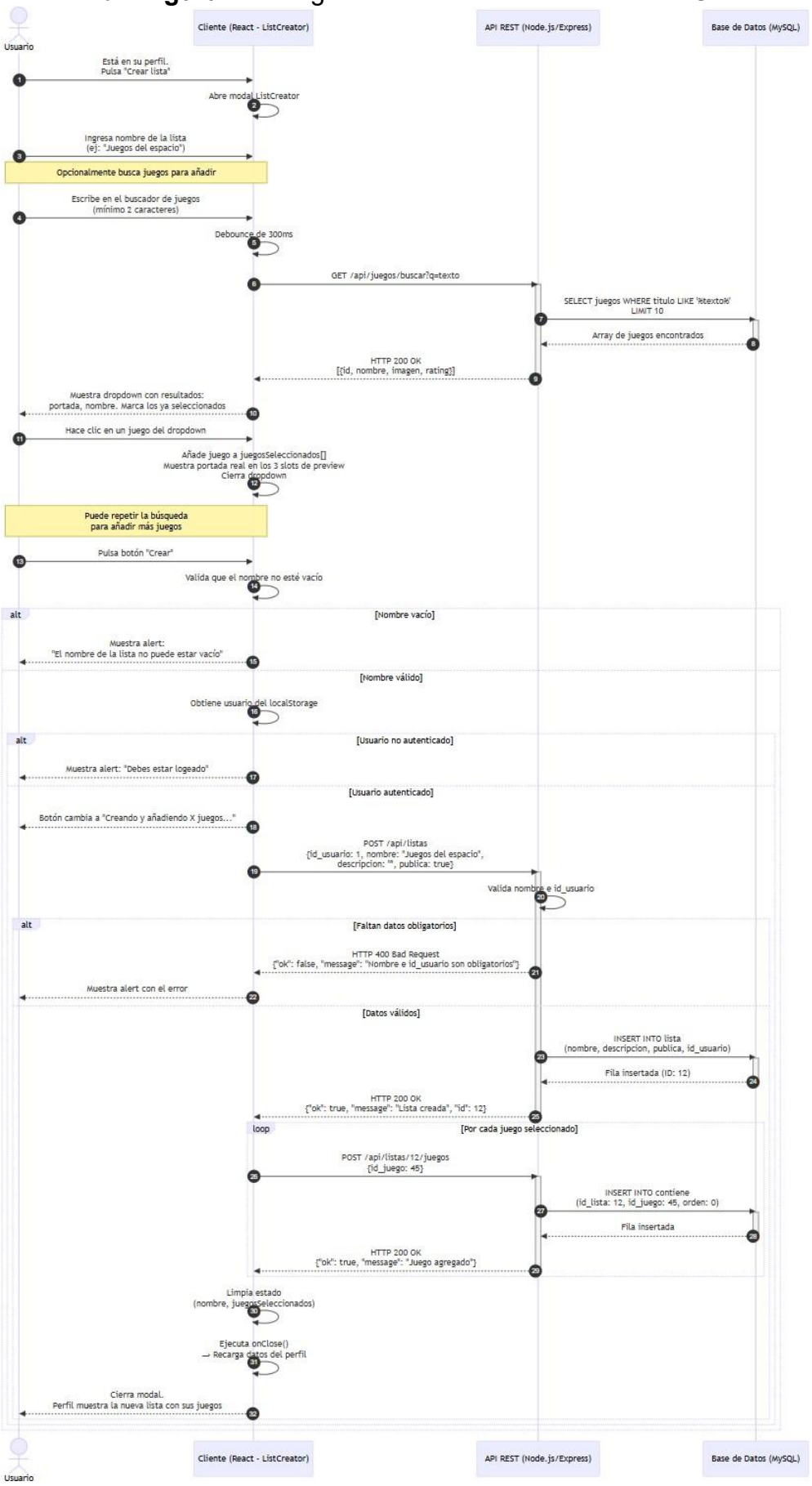
- **Tabla 6.** Tabla de especificación de requisitos referente a Añadir juego a biblioteca.

NOMBRE DEL CASO	Añadir juego a biblioteca de usuario
DESCRIPCION	El usuario categoriza un juego en su biblioteca personal pulsando uno de los botones de estado (Wishlist, Jugando, Completado).
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe estar logueado
SECUENCIA NORMAL	1. El usuario hace clic en el botón del estado deseado. 2. El sistema verifica que existe una sesión activa. 3. El sistema realiza una petición POST a /api/usuarios/{id}/juegos con el ID del juego y el estado. 4. El servidor actualiza el registro en la BBDD. 5. El servidor responde afirmativamente. 6. El sistema muestra un mensaje de éxito ("Añadido como...").
POSTCONDICIONES	El juego queda registrado en la biblioteca del usuario con su nuevo estado.
EXCEPCIONES	- **Usuario no logueado:** El sistema detiene el flujo y muestra el error *"Debes estar logeado"*. No se realiza petición al servidor. - **Error del servidor / Red:** El servidor devuelve error o falla la conexión. El sistema captura la excepción y muestra el error correspondiente.

- **Figuras 13 y 14 y Tabla 7.** Crear Lista.
  - **Figura 13.** Diagrama de Actividad referente a Crear Lista.



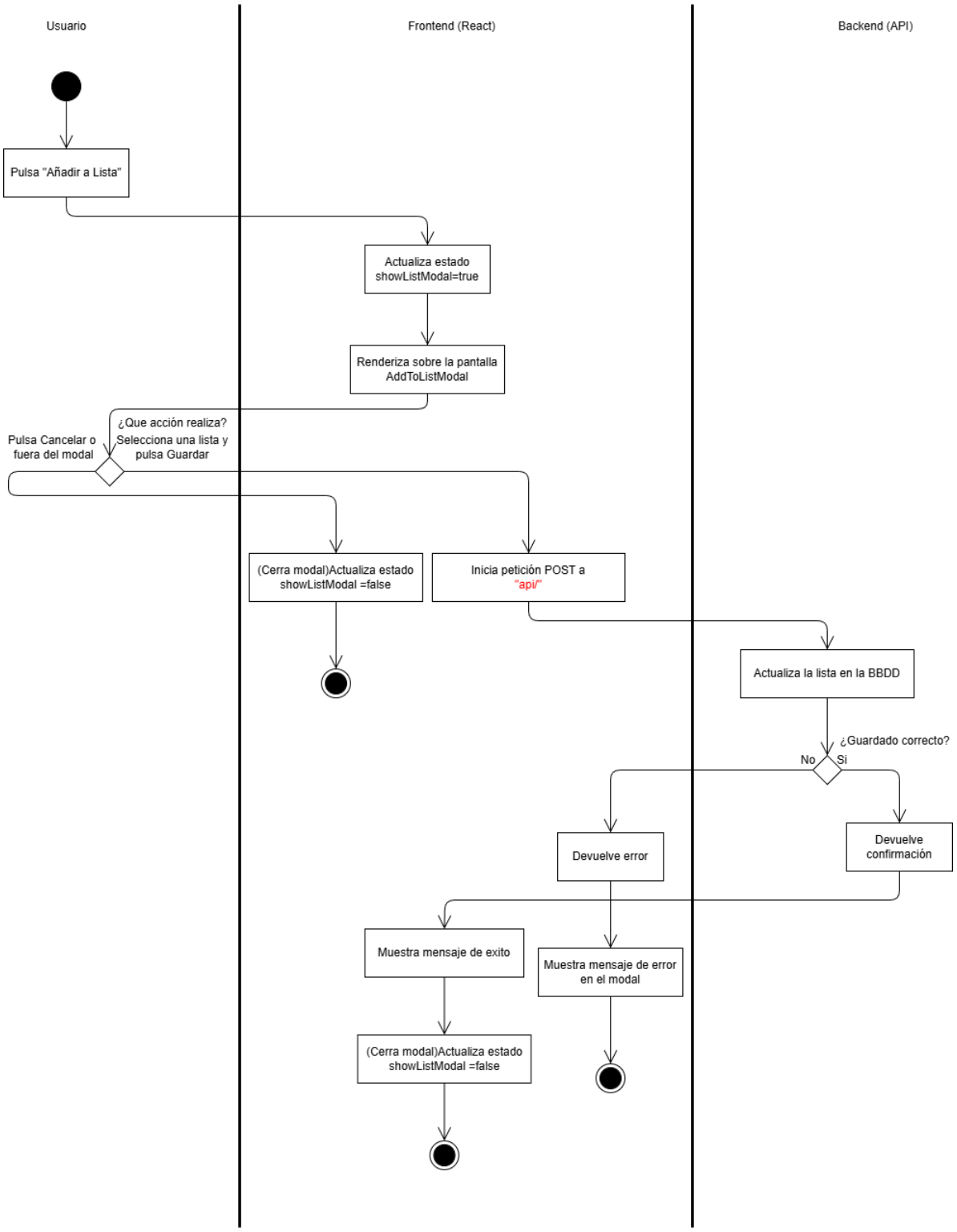
- **Figura 14.** Diagrama de Secuencia referente a Crear Lista.



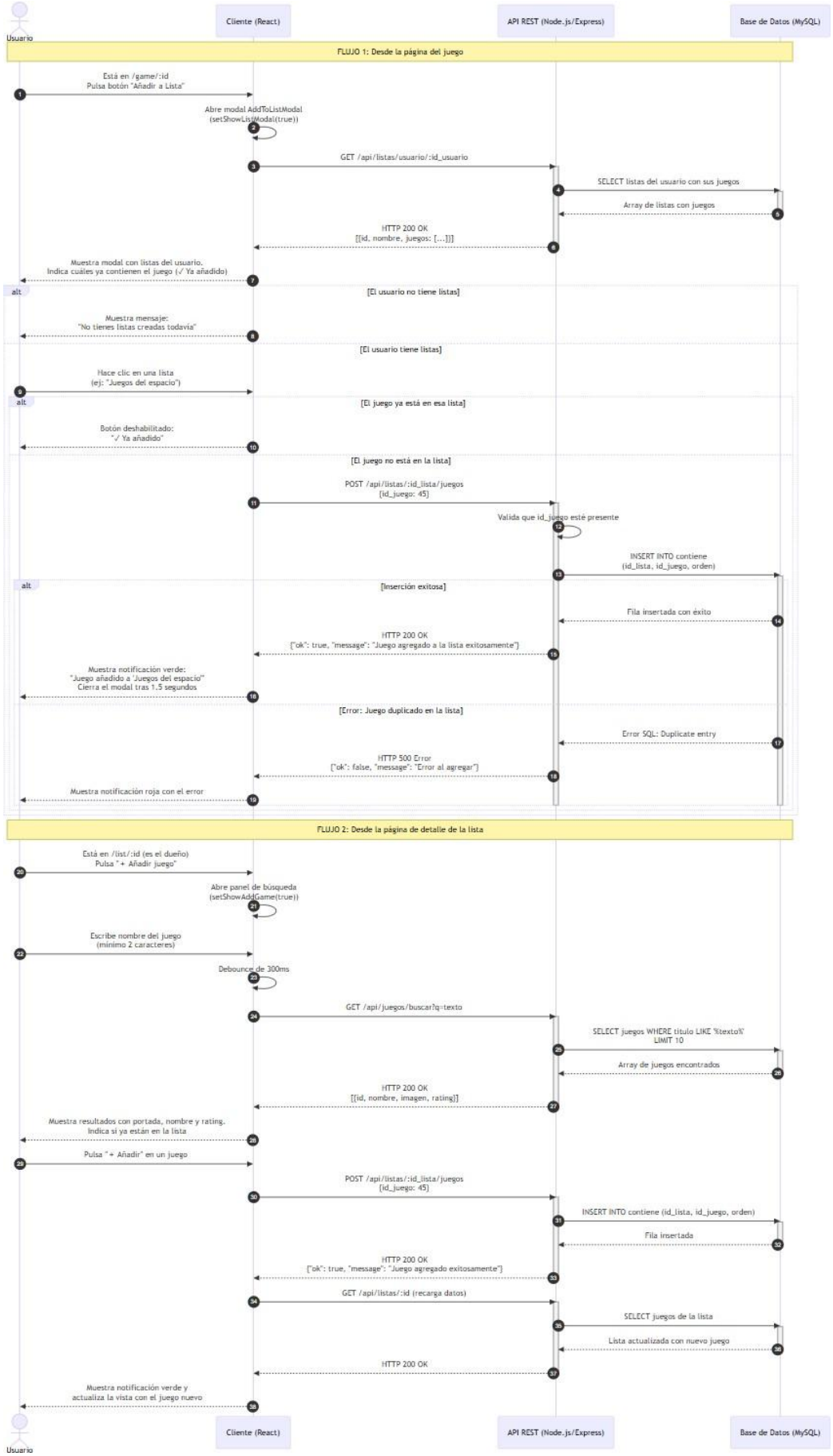
- **Tabla 7.** Tabla de especificación de requisitos referente a Crear Lista.

NOMBRE DEL CASO	Crear Lista
DESCRIPCION	Un usuario autenticado crea una nueva lista de videojuegos desde la pestaña "Listas" de su propio perfil.
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe estar logueado y estar visualizando su propio perfil
SECUENCIA NORMAL	1. El usuario navega a la pestaña "Listas" y pulsa "Crear lista". 2. El sistema despliega el modal. 3. El usuario introduce los datos de la lista y pulsa "Guardar". 4. El sistema envía una petición de creación al servidor. 5. El servidor registra la nueva lista en la base de datos y responde con éxito. 6. 6. El sistema cierra el modal. 7. 7. El sistema ejecuta una recarga automática de los datos del perfil (loadProfileData). 8. 8. El servidor devuelve las listas actualizadas y el sistema las renderiza en pantalla.
POSTCONDICIONES	La lista se ha creado en la BBDD, vinculada al usuario y aparece en la UI.
EXCEPCIONES	- **Cierre voluntario:** El usuario pulsa "Cancelar" o cierra el modal. El estado isModalOpen pasa a false y no se hace petición a la API. - **Error al guardar:** Si la API devuelve un error, el componente muestra el error y el modal no se cierra para permitir correcciones.

- **Figuras 15 y 16 y Tabla 8.** Añadir juego a lista.
  - **Figura 15.** Diagrama de Actividad referente a Añadir juego a lista.



- **Figura 16.** Diagrama de Secuencia referente a Añadir juego a lista.

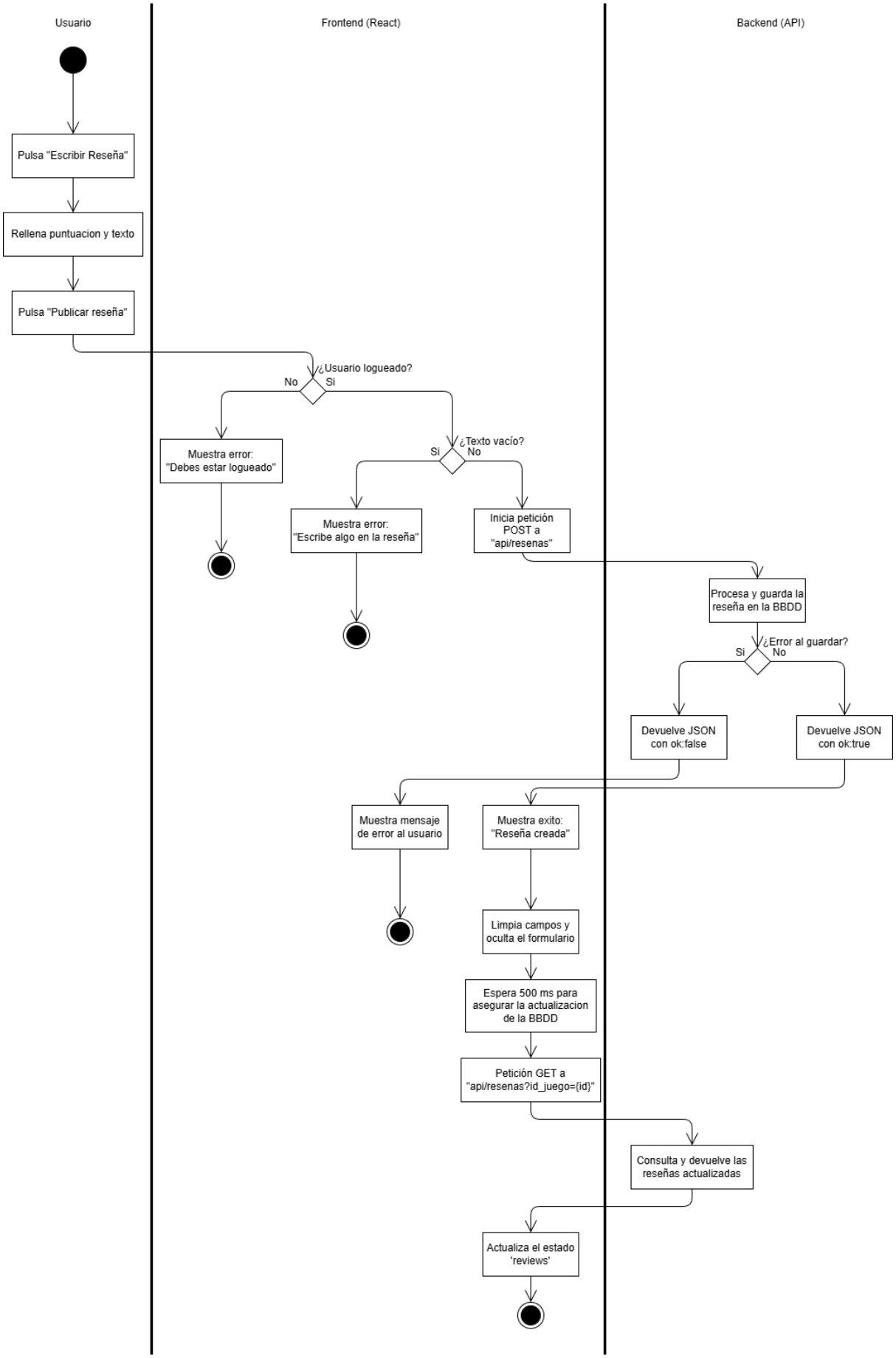


- **Tabla 8.** Tabla de especificación de requisitos referente a Añadir juego a lista.

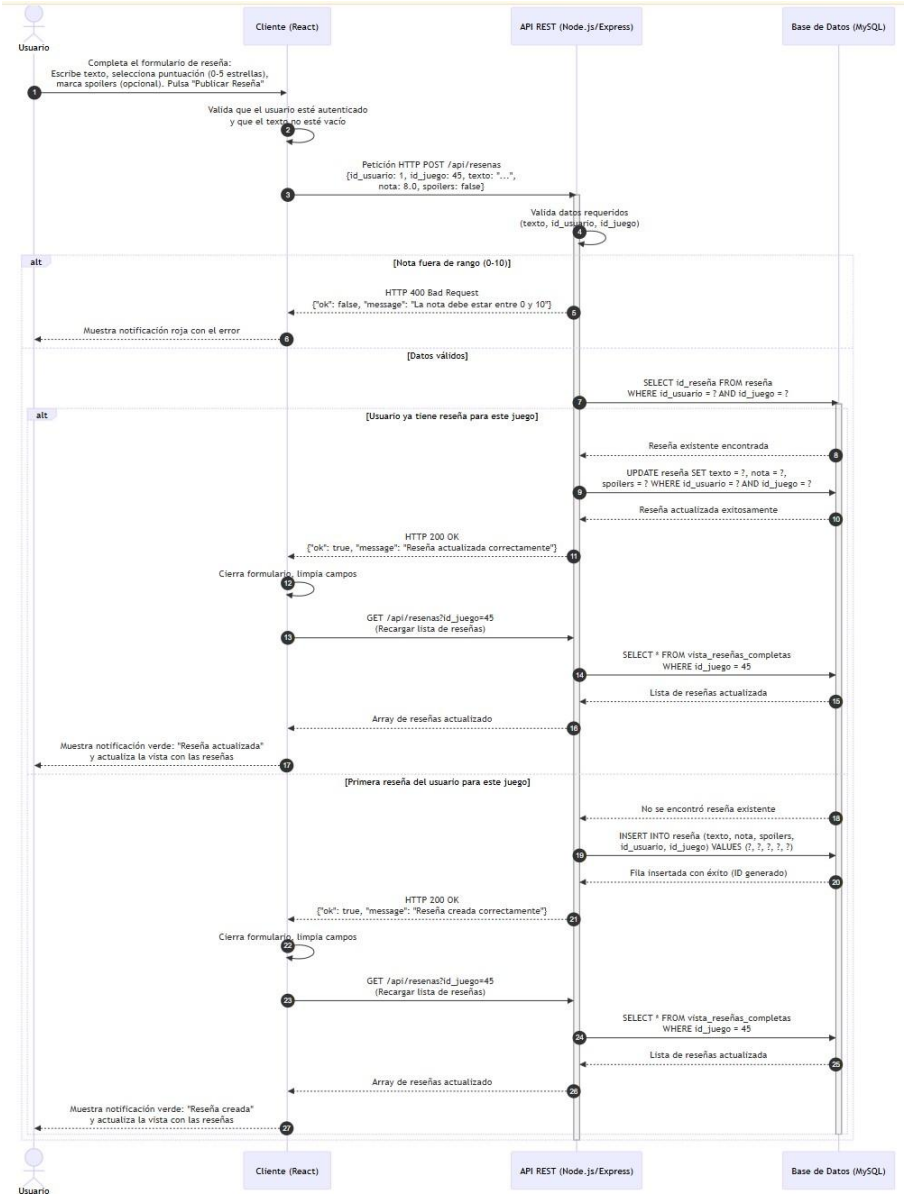
NOMBRE DEL CASO	Añadir juego a lista
DESCRIPCION	El usuario abre el modal de listas para añadir el juego actual a una de sus listas personalizadas creadas previamente.
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe estar logueado
SECUENCIA NORMAL	1. El usuario pulsa "Añadir a Lista". 2. El sistema renderiza el modal superpuesto. 3. El usuario selecciona una lista de las disponibles en el modal. 4. El sistema envía una petición al servidor para vincular el juego con la lista elegida. 5. El servidor confirma la operación. 6. El sistema cierra el modal y muestra un mensaje de éxito.
POSTCONDICIONES	El juego queda vinculado a la lista seleccionada por el usuario.
EXCEPCIONES	- **Cierre voluntario:** El usuario pulsa fuera del modal o en el botón de cerrar. El sistema desmonta el modal sin hacer cambios. - **Error al guardar:** Si la API falla, el modal muestra un error en lugar de cerrarse.

Bloque C: Interacción Social

- **Figuras 17 y 18 y Tabla 9.** Publicar review.
    - **Figura 17.** Diagrama de Actividad referente a Publicar review.


Usuario
Frontend (React)
Backend (API)
Pulsa "Escribir Reseña"
Rellena puntuacion y texto
Pulsa "Publicar reseña"
¿Usuario logueado?
No
Si
Muestra error: "Debes estar logueado"
¿Texto vacío?
Si
No
Muestra error: "Escribe algo en la reseña"
Inicia petición POST a "api/resenas"
Procesa y guarda la reseña en la BBDD
¿Error al guardar?
Si
No
Devuelve JSON con ok:false
Devuelve JSON con ok:true
Muestra mensaje de error al usuario
Muestra exito: "Reseña creada"
Limpia campos y oculta el formulario
Espera 500 ms para asegurar la actualizacion de la BBDD
Petición GET a "api/resenas?id_juego={id}"
Consulta y devuelve las reseñas actualizadas
Actualiza el estado 'reviews'

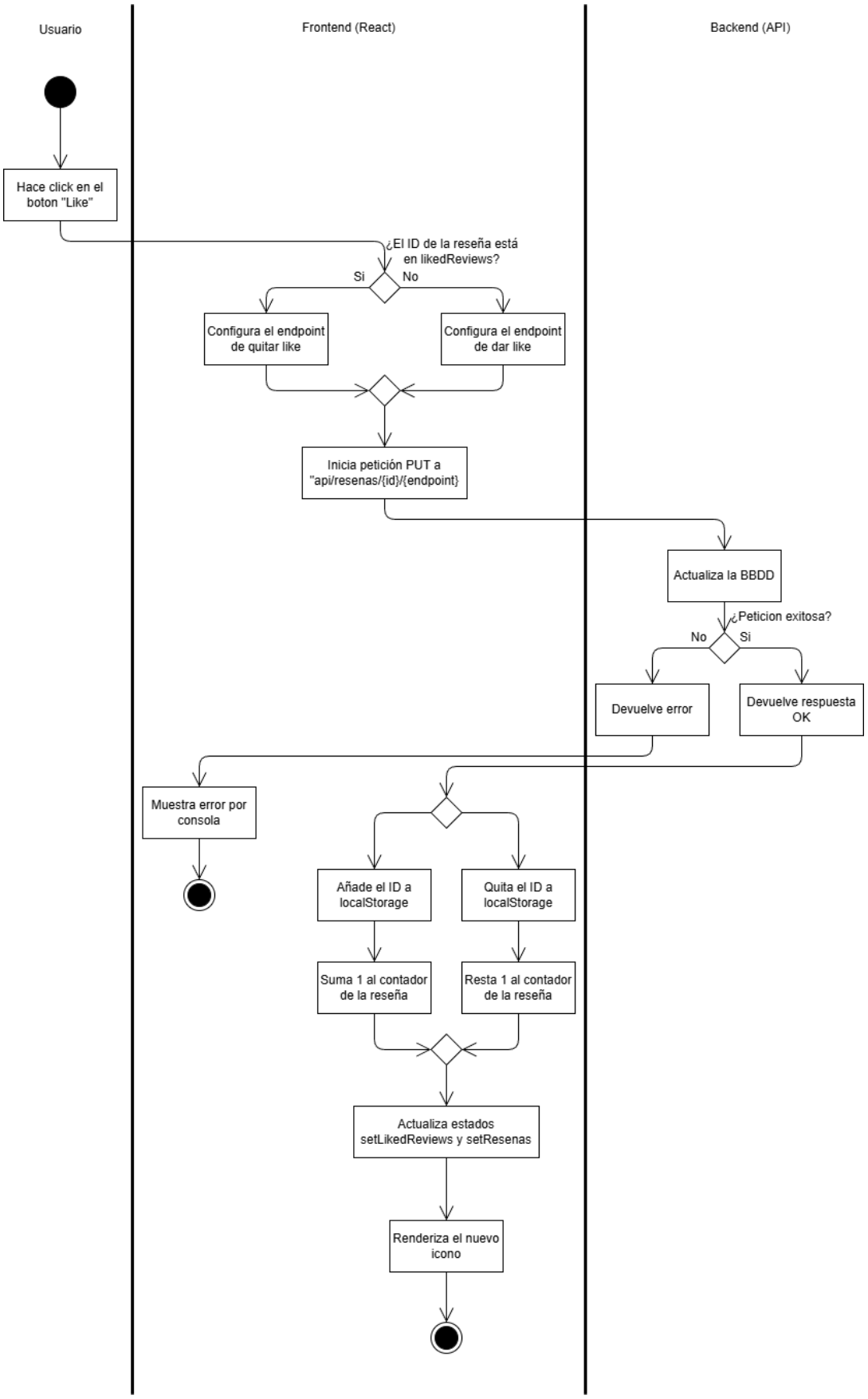
- **Figura 18.** Diagrama de Secuencia referente a Publicar review.



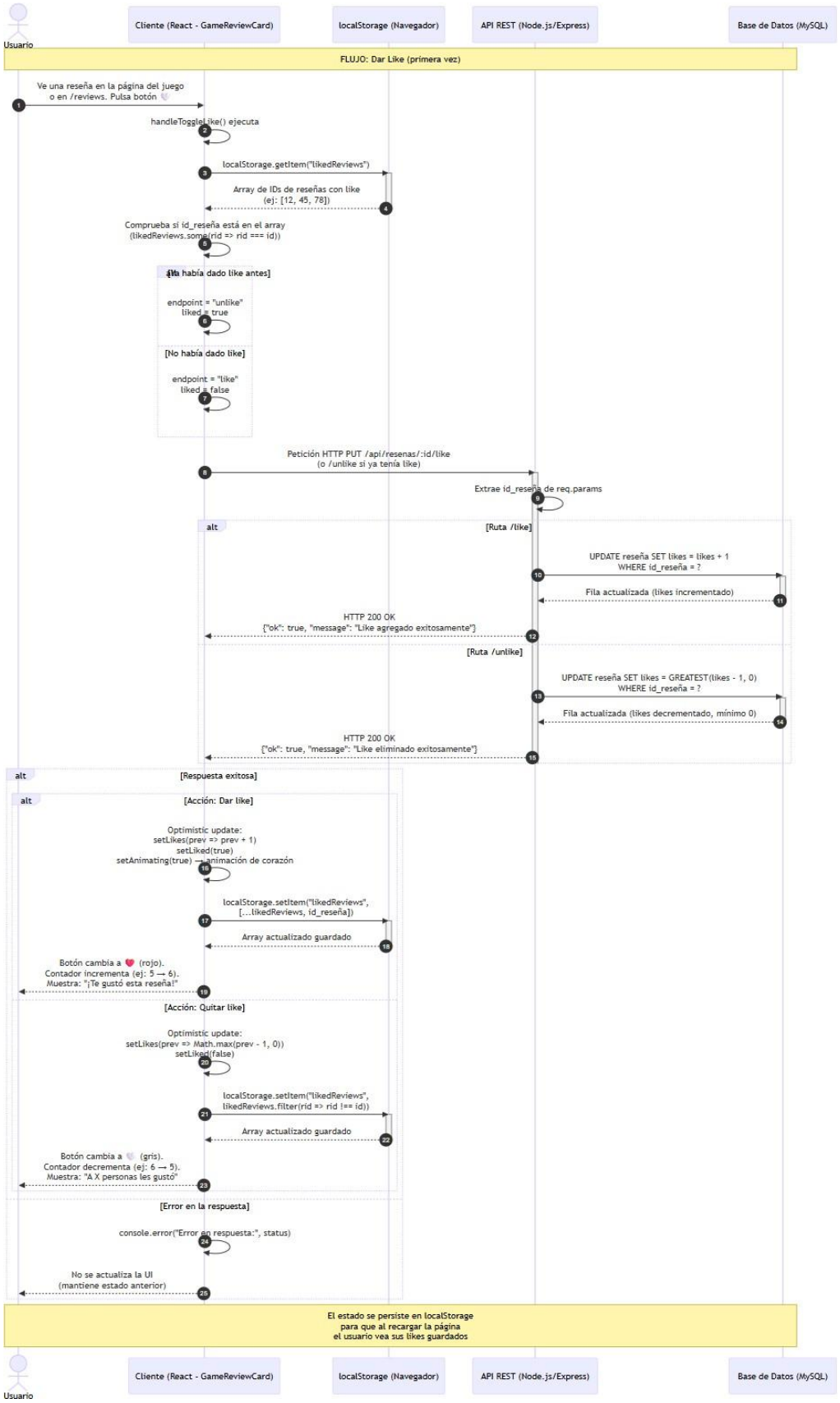
- **Tabla 9.** Tabla de especificación de requisitos referente a Publicar review.

NOMBRE DEL CASO	Publicar review de un juego
DESCRIPCION	Un usuario registrado y con sesión activa valora y escribe una opinión sobre un juego en su página correspondiente
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe haber iniciado sesión y encontrarse en la página de detalles de un juego
SECUENCIA NORMAL	1. El usuario pulsa "Escribir Reseña" para desplegar el formulario. 2. Introduce una puntuación (estrellas) y el texto de la reseña. 3. Pulsa "Publicar Reseña". 4. El sistema valida que el usuario esté logueado y que el texto no esté vacío. 5. El sistema envía los datos al servidor. 6. El servidor guarda la reseña en la BBDD y responde con éxito. 7. El sistema muestra "Reseña creada", limpia y oculta el formulario. 8. Tras 500ms, el sistema vuelve a solicitar al servidor la lista de reseñas actualizada para mostrarla en pantalla.
POSTCONDICIONES	La reseña queda vinculada al juego y al usuario

- **Figuras 19 y 20 y Tabla 10.** Dar/Quitar like a review.
  - **Figura 19.** Diagrama de Actividad referente a Dar/Quitar like a review.



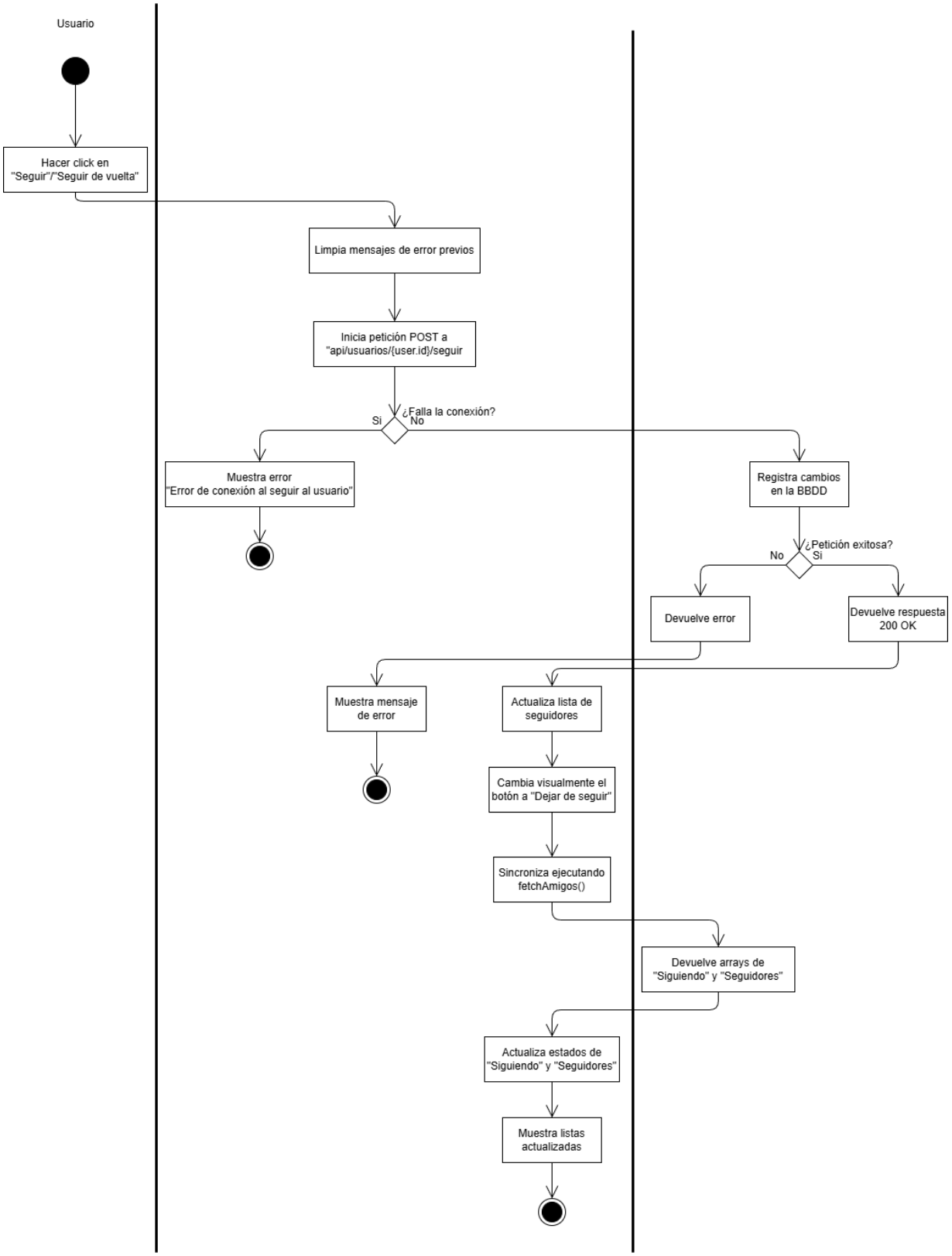
- **Figura 20.** Diagrama de Secuencia referente a Dar/Quitar like a review.



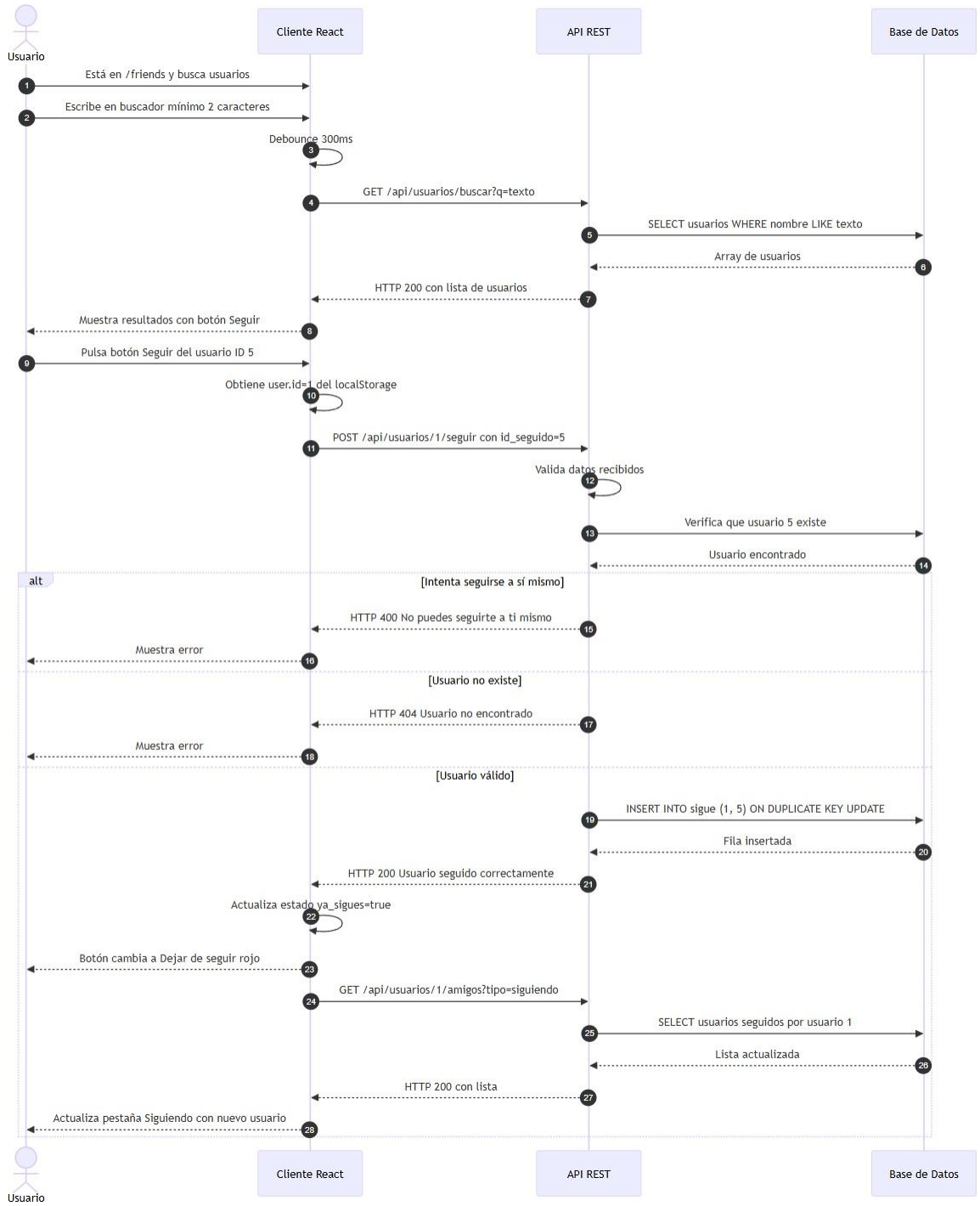
- **Tabla 10.** Tabla de especificación de requisitos referente a Dar/Quitar like a review.

NOMBRE DEL CASO	Dar/quitar like a una review
DESCRIPCION	El usuario interactúa con el botón de "Me gusta" de una reseña para mostrar su aprecio o retirarlo.
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe visualizar la review.
SECUENCIA NORMAL	1. El usuario hace clic en el botón de Like de una reseña específica. 2. El sistema comprueba en el almacenamiento local si el usuario ya le había dado like previamente. 3. El sistema determina el *endpoint* correcto (/like o /unlike). 4. El sistema envía una petición PUT al servidor. 5. El servidor actualiza el contador en la base de datos y responde afirmativamente. 6. El sistema actualiza el registro local guardando o eliminando el ID de la reseña en localStorage. 7. El sistema actualiza el estado sumando o restando 1 al contador visualmente e invirtiendo el icono al instante.
POSTCONDICIONES	El contador de la reseña se actualiza en el servidor y la interfaz refleja el cambio sin recargar la página.
EXCEPCIONES	- **Error del servidor / Red:** Si la petición falla, el sistema captura el error por consola, pero la interfaz gráfica no se altera (se cancela el proceso).

- **Figuras 21 y 22 y Tabla 11.** Seguir a otro usuario.
    - **Figura 21.** Diagrama de Actividad referente a Seguir a otro usuario.



- **Figura 22.** Diagrama de Secuencia referente a Seguir a otro usuario.

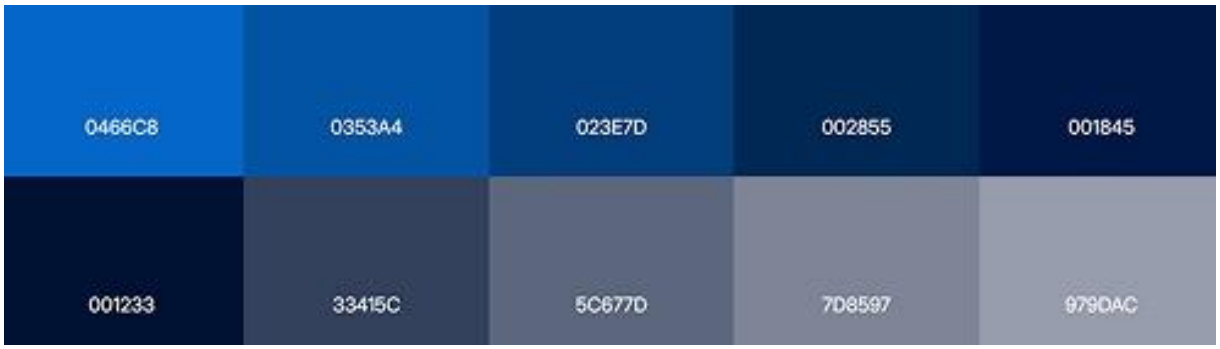


- **Tabla 11.** Tabla de especificación de requisitos referente a Seguir a otro usuario.

NOMBRE DEL CASO	Seguir a otro usuario registrado
DESCRIPCION	Un usuario autenticado busca a otro usuario en la plataforma y pulsa el botón para empezar a seguirlo.
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe haber iniciado sesión y estar en la pantalla de "Amigos" (/friends).
SECUENCIA NORMAL	1. El usuario introduce el nombre en el buscador o visualiza su lista de seguidores. 2. El usuario hace clic en el botón "Seguir" junto al nombre de otro usuario. 3. El sistema limpia errores previos y realiza una petición POST enviando el ID del usuario a seguir. 4. El servidor registra la relación (seguidor-seguido) en la base de datos y responde con éxito. 5. El sistema actualiza el estado local cambiando el estado del botón a "Dejar de seguir". 6. El sistema recarga la lista de "Siguiendo" y "Seguidores" actualizada desde el servidor.
POSTCONDICIONES	El usuario pasa a seguir al usuario seleccionado y las listas de la interfaz se actualizan.
EXCEPCIONES	- **Usuario no logueado:** Si no hay un user.id, el sistema muestra el error *"Debes iniciar sesión..."* al cargar la página. - **Error de servidor/red al seguir:** Si la petición POST falla, el sistema captura el error y muestra el mensaje.

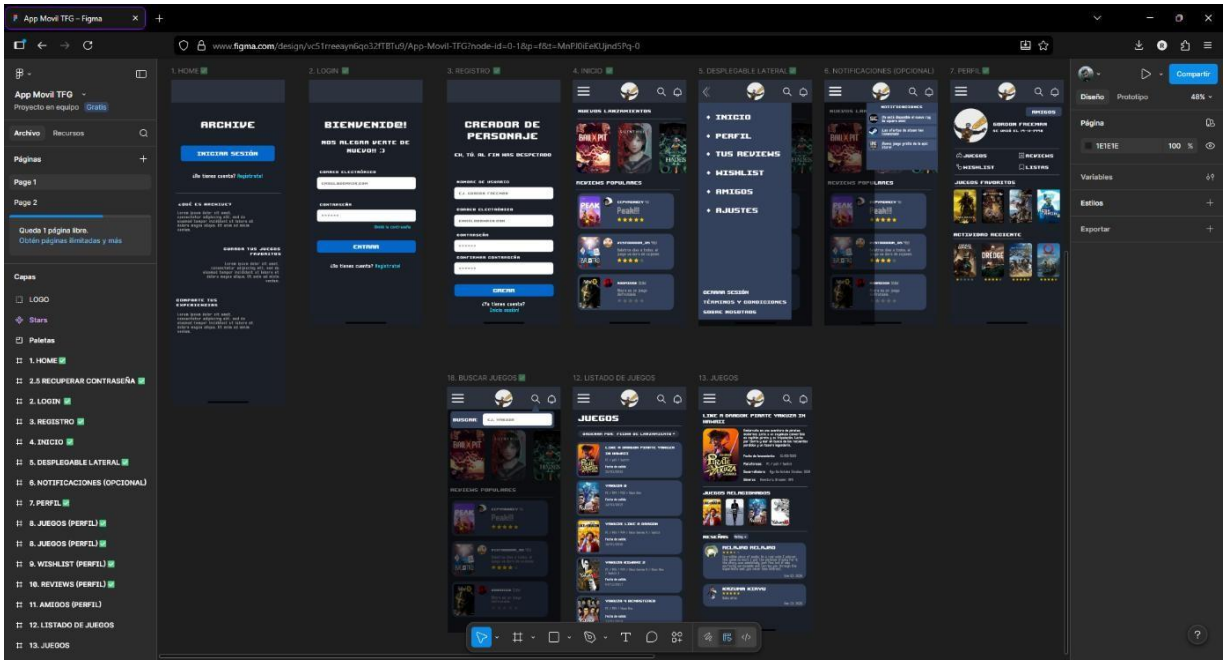
Anexo 3. Diseño Visual y Experiencia de Usuario (UX/UI)

- **Imagen 1.** Paleta de Colores.



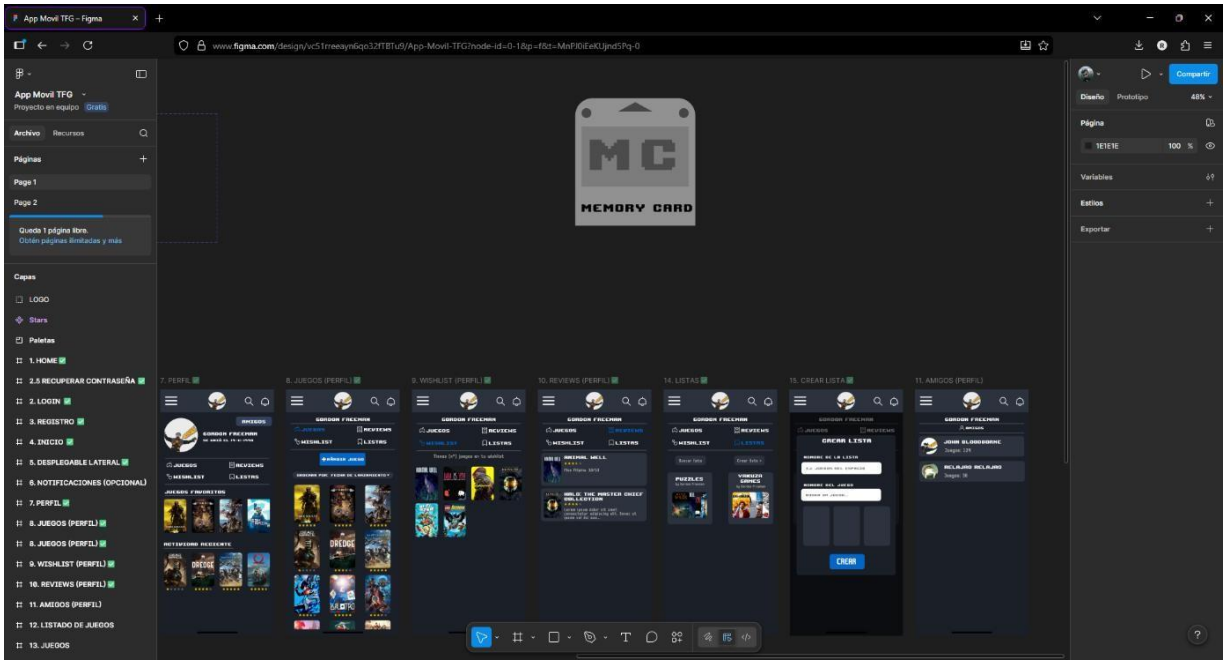
Descripción: Definición de la identidad visual diseñada para resaltar elementos gráficos como carátulas y reducir la carga visual.

- **Figura 2.** Diseños de Interfaz Móvil en Figma I.



Descripción: Prototipos visuales de los diseños de los apartados de la aplicación móvil (Gestión de usuarios, buscador de juegos y plantilla para las páginas de juegos).

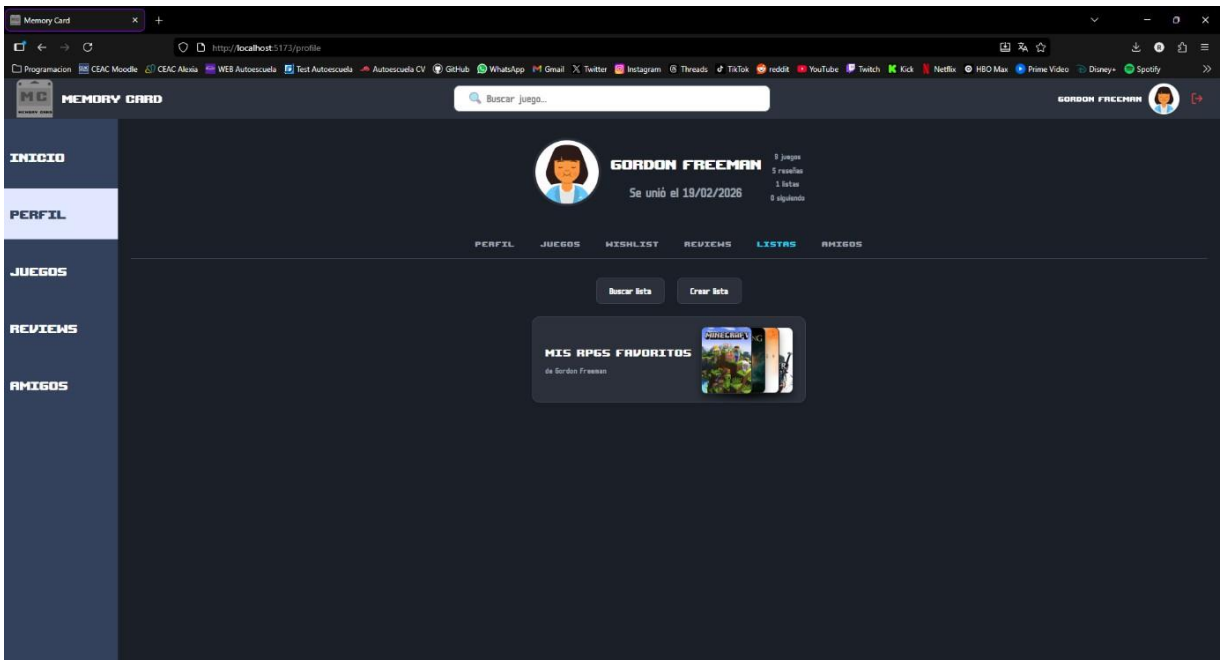
- **Figura 3.** Diseños de Interfaz Móvil en Figma II.



Descripción: Prototipos visuales de los diseños de los apartados de la aplicación móvil (Gestión de listas, plantillas para las páginas de perfiles e icono/logo).

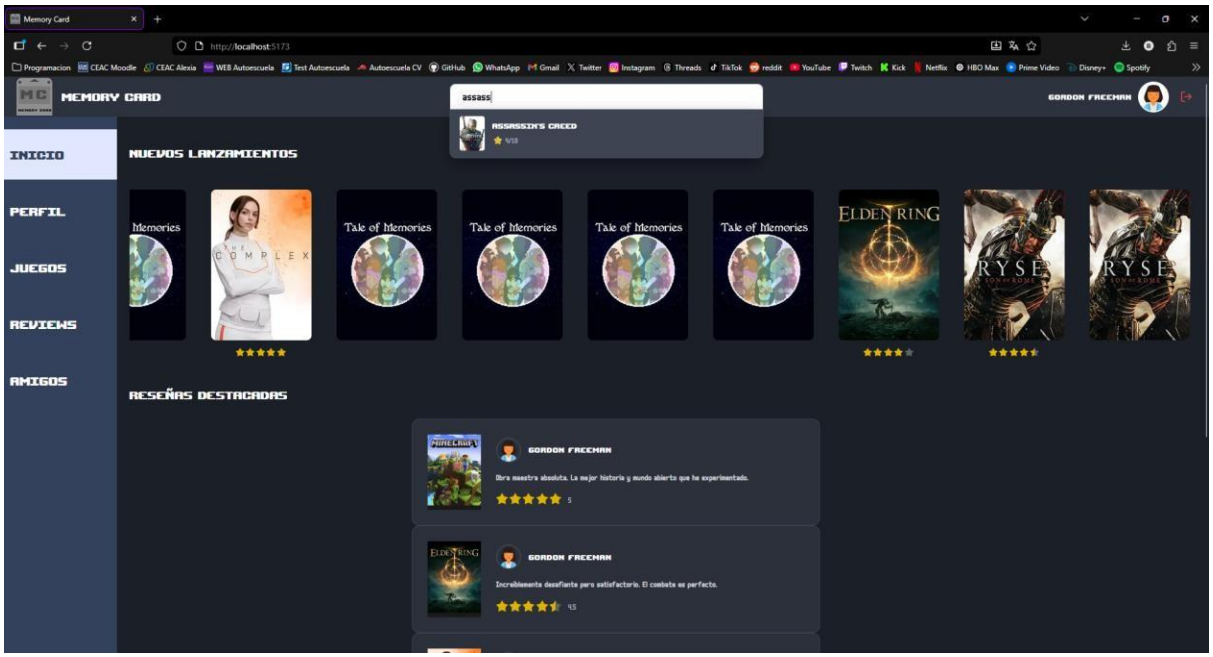
Anexo 4. Implementación y Sincronización

- **Figura 1. Interfaz del Cliente Web (React) - Perfil.**



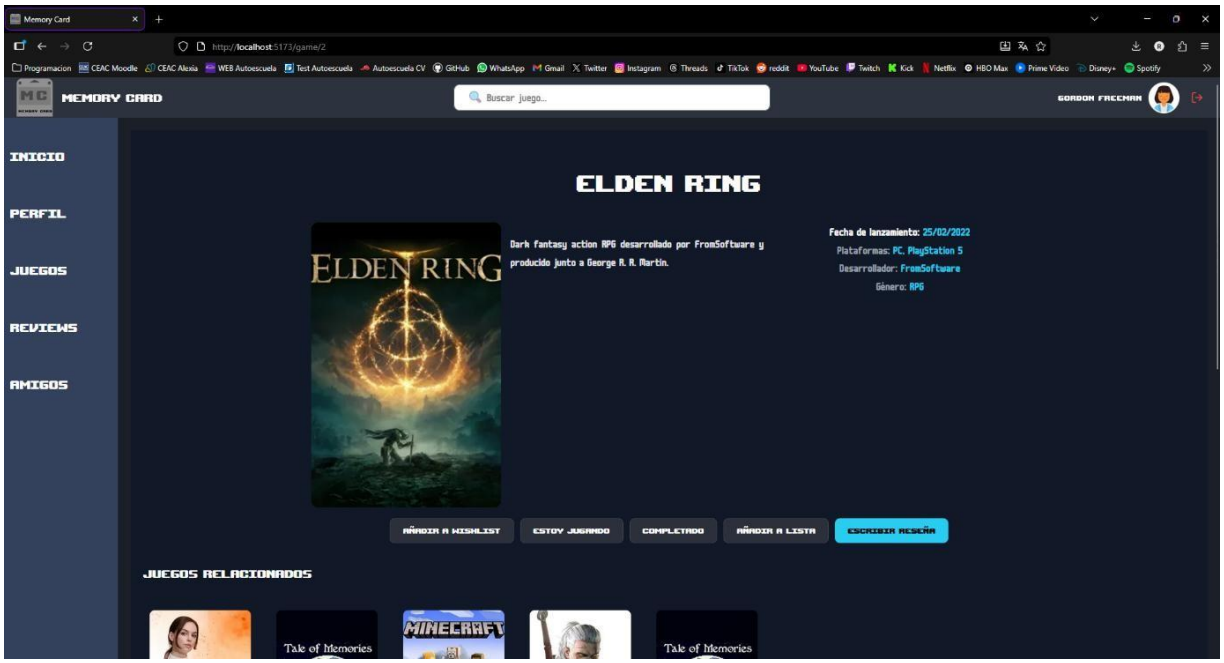
Descripción: Captura del Cliente Web en el apartado de listas del perfil.

- **Figura 2. Interfaz del Cliente Web (React) - Inicio.**



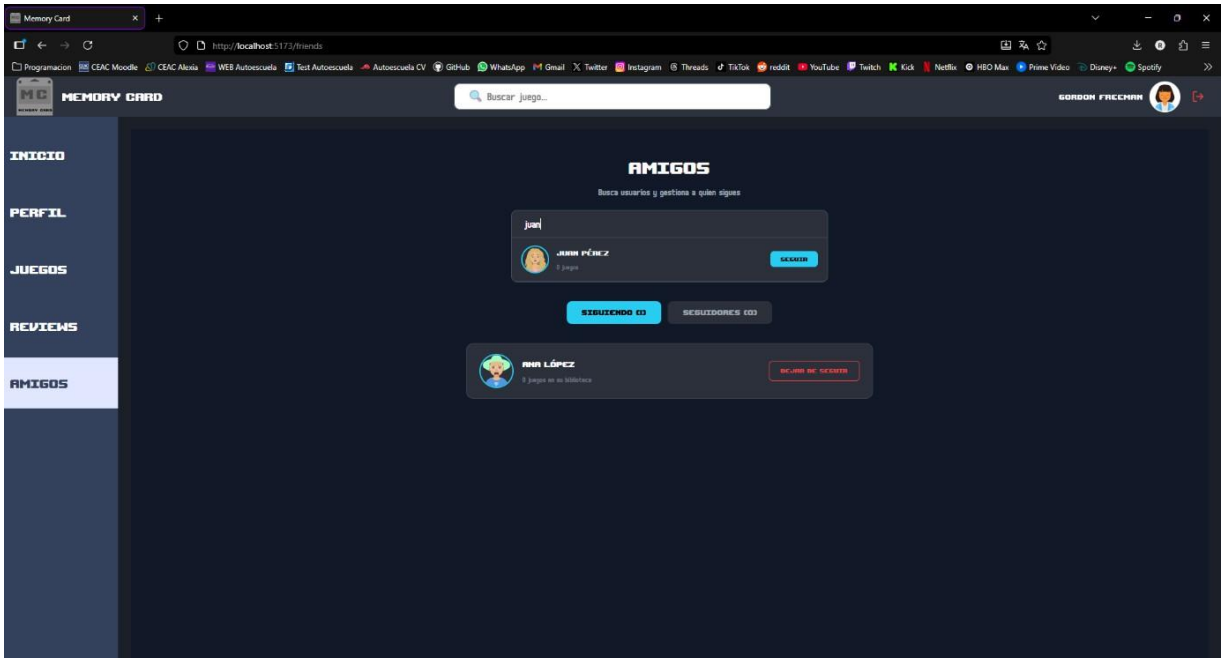
Descripción: Captura del Cliente Web en la página principal, mientras se prueba el buscador.

- **Figura 3. Interfaz del Cliente Web (React) - Juego.**



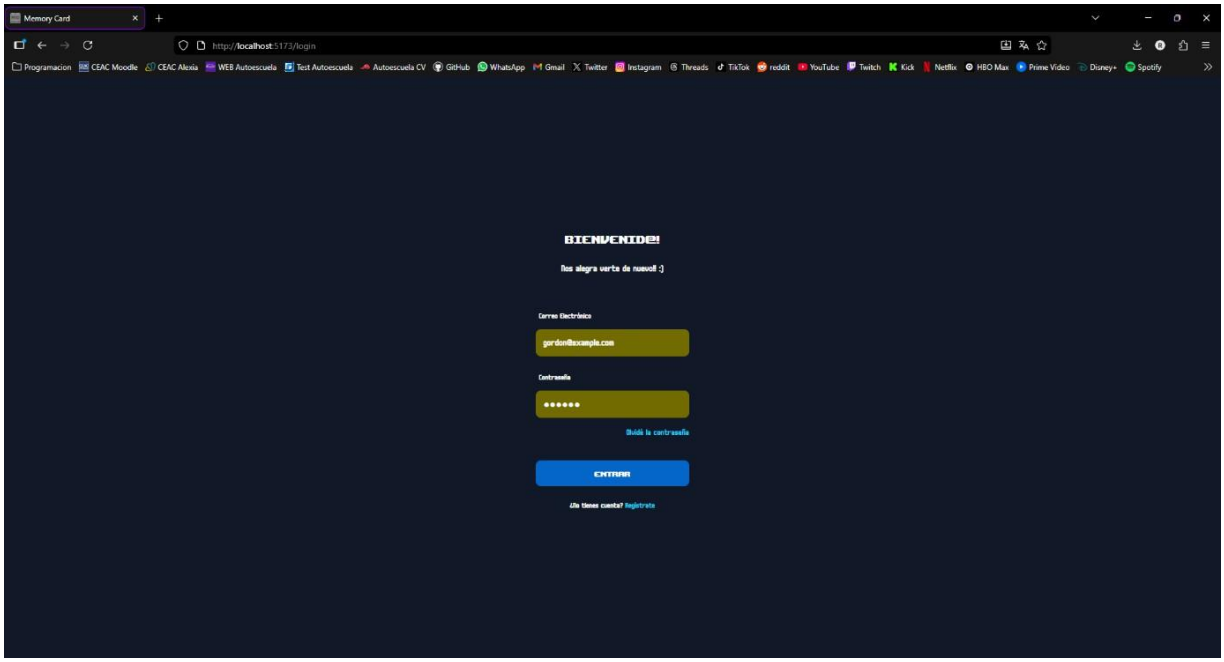
Descripción: Captura del Cliente Web en la página correspondiente a uno de los juegos.

- **Figura 4. Interfaz del Cliente Web (React) - Amigos.**



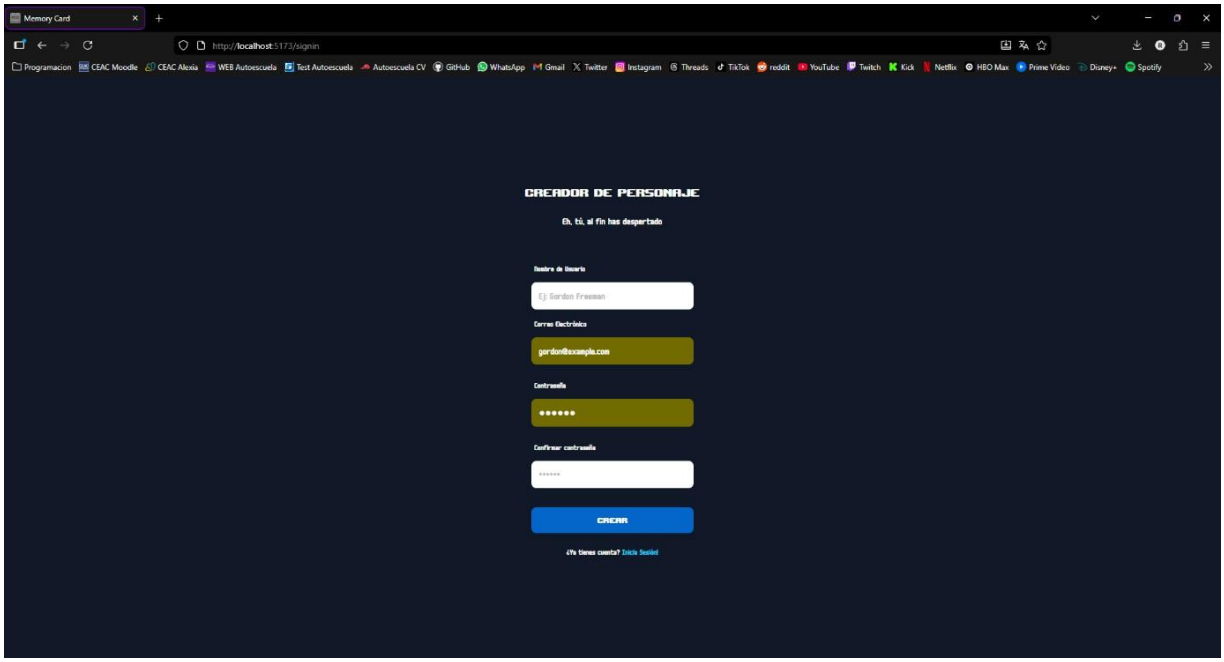
Descripción: Captura del Cliente Web en la página de amigos mientras se usa el buscador.

- **Figura 5. Interfaz del Cliente Web (React) – Inicio de Sesión.**



Descripción: Captura del Cliente Web en la página de inicio de sesión.

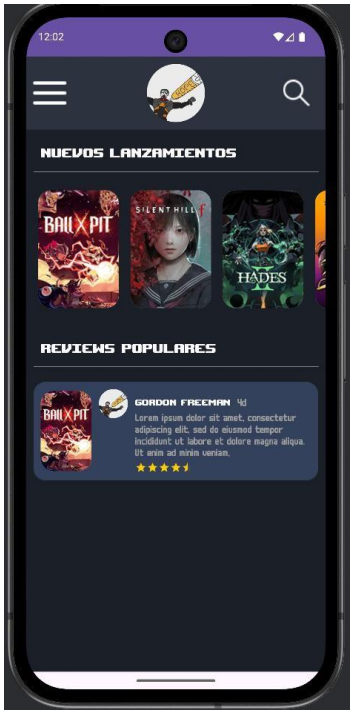
- **Figura 6. Interfaz del Cliente Web (React) - Registro.**



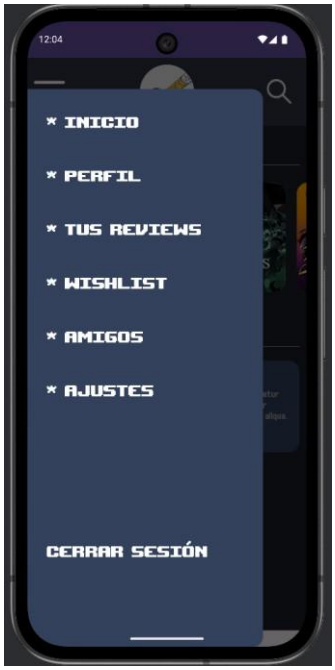
Descripción: Captura del Cliente Web en la página de registro.

- **Figura 7. Inicio de sesión en la Aplicación Móvil Nativa (Android).**

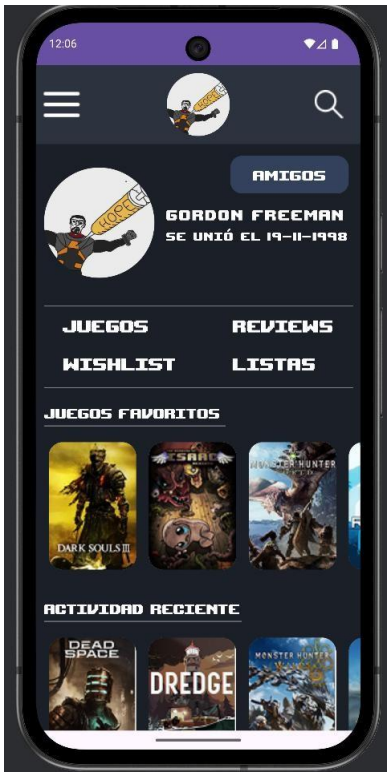
- **Figura 8. Página principal en la Aplicación Móvil Nativa (Android).**



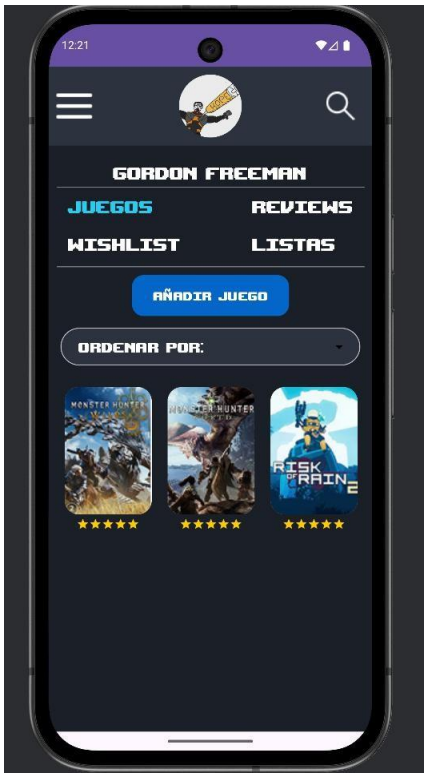
- **Figura 9. Menú desplegable lateral en la Aplicación Móvil Nativa (Android).**



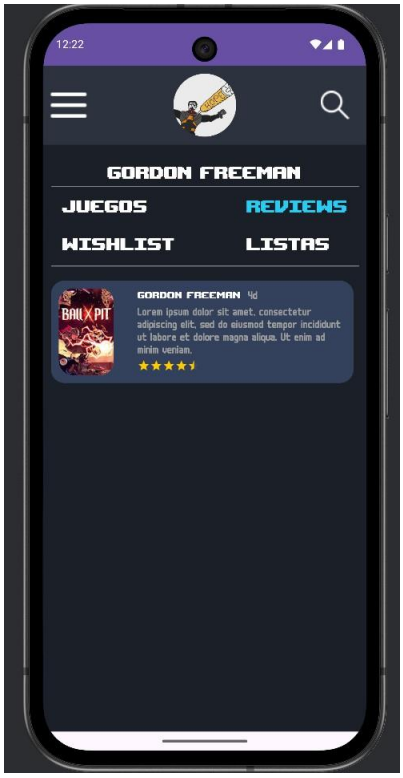
- **Figura 10. Sección principal del perfil en la Aplicación Móvil Nativa (Android).**



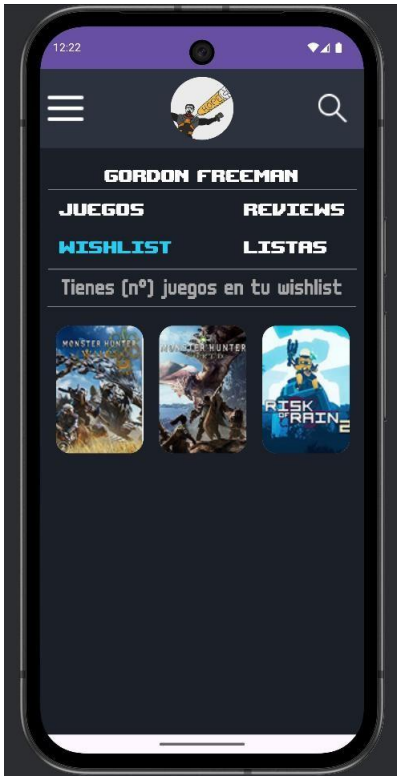
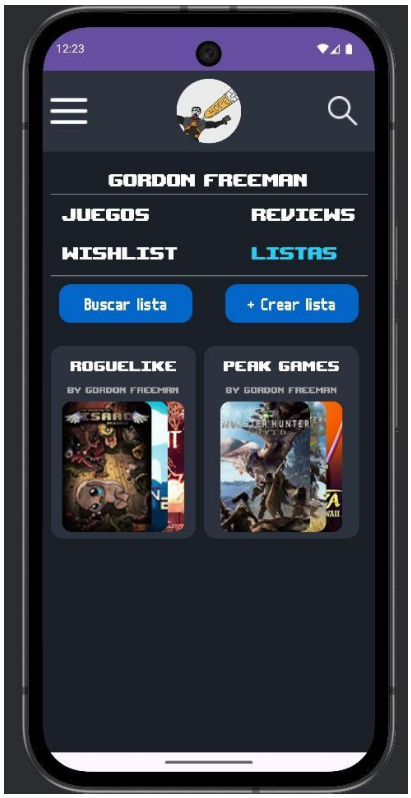
- **Figura 11. Sección de Biblioteca del perfil en la Aplicación Móvil Nativa (Android).**



- **Figura 12. Sección de reviews del perfil en la Aplicación Móvil Nativa (Android).**



- **Figura 13. Sección de Lista de deseos del perfil en la Aplicación Móvil Nativa (Android).**



- **Figura 14. Sección de listas personalizadas del perfil en la Aplicación Móvil Nativa (Android).**